SAMURAI Robot

Математика, Физика и Дифференциальные системы

Технический справочник по математическому ядру проекта

Фильтры состояния:	Extended Kalman Filter (2D velocity, IMU yaw)
Оценка позиции:	Акселерометр + одометрия, слияние через EKF
Управление:	Pure Pursuit, пропорциональный регулятор
Картография:	SLAM на log-odds, трассировка лучей
Симулятор:	Кинематика, пересечение луч–сфера, перспектива
Алгоритмы:	A*, Bresenham, проекции, кватернионы

March 26, 2026

Contents

Введение **6**

1 Теоретические основы систем управления **7**

1.1 Модель в пространстве состояний . . . 7
1.1.1 Применение в проекте SAMURAI . . . 7
1.2 Линеаризация нелинейных систем . . . 8
1.2.1 Пример: транспортная тележка . . . 8
1.2.2 Метод линеаризации по Тейлору (матрица Якоби) . . . 8
1.2.3 Пример: якобиан для транспортной тележки . . . 9
1.2.4 Связь с EKF в проекте SAMURAI . . . 9
1.3 Передаточная функция . . . 9
1.3.1 Характеристический полином и резольвента . . . 9
1.3.2 Алгоритм Фаддеева–Леверье . . . 10
1.3.3 Уравнение «вход–выход» . . . 10
1.4 Устойчивость систем управления . . . 10
1.4.1 Определение устойчивости . . . 10
1.4.2 Критерий собственных значений . . . 10
1.4.3 Критерий Гурвица . . . 11
1.4.4 Решение уравнения состояния . . . 11
1.4.5 Жорданова форма и матричная экспонента . . . 11
1.5 Дискретные системы и фильтр Калмана . . . 11
1.5.1 Дискретная модель в пространстве состояний . . . 11
1.5.2 Общие уравнения фильтра Калмана . . . 12
1.5.3 Расширенный фильтр Калмана (EKF) . . . 12
1.6 Модель электродвигателя постоянного тока . . . 13
1.6.1 Пространство состояний для двигателя . . . 13
1.6.2 Передаточная функция двигателя . . . 13
1.7 Пропорциональный регулятор (P-регулятор) . . . 14
1.7.1 Закон управления . . . 14
1.7.2 Передаточная функция замкнутой системы . . . 14
1.7.3 Статическая ошибка . . . 14
1.7.4 Применение P-регуляторов в проекте . . . 14
1.8 Переходная характеристика . . . 14
1.9 Управляемость и наблюдаемость . . . 15
1.9.1 Матрица управляемости . . . 15
1.9.2 Матрица наблюдаемости . . . 15
1.10 Сводная таблица: теория → реализация . . . 15

2 2D Velocity Extended Kalman Filter **16**
2.1 Постановка задачи . . . 16
2.2 Вектор состояния и модель . . . 16
2.2.1 Вектор состояния . . . 16
2.2.2 Матрица ковариации . . . 16
2.3 Шаг предсказания (Prediction) . . . 17
2.4 Шаг коррекции (Update) по акселерометру . . . 18
2.5 ZUPT — обновление нулевой скорости . . . 19
2.6 Неголономное ограничение . . . 20
2.7 Модуль скорости . . . 20
2.8 Теоретическое обоснование: связь с ТАУ . . . 20
2.8.1 Модель в пространстве состояний . . . 20
2.8.2 Устойчивость фильтра . . . 21
2.8.3 Оптимальность усиления Калмана . . . 21
2.9 Сводная таблица параметров . . . 21

3 IMU EKF: одномерный фильтр Калмана для угла курса **22**
3.1 Постановка задачи . . . 22
3.2 Вектор состояния . . . 22
3.3 Шаг предсказания: интегрирование гироскопа . . . 23
3.4 Шаг коррекции: крен и тангаж из акселерометра . . . 24
3.5 Теоретическое обоснование: линеаризация и устойчивость . . . 25
3.5.1 Нелинейная модель и якобиан . . . 25
3.5.2 Устойчивость фильтра . . . 25
3.5.3 Наблюдаемость . . . 25
3.5.4 Связь с передаточной функцией . . . 26
3.6 Сводная таблица параметров . . . 26

4 Оценка позиции по акселерометру с слиянием датчиков **27**
4.1 Архитектура системы слияния датчиков . . . 27
4.2 Константы физики . . . 27
4.3 Шаг 1: Вычитание гравитации . . . 28
4.4 Шаг 2: Поворот из тела в мировую СК . . . 29
4.5 Шаг 3: Компенсация эффекта Кориолиса . . . 29
4.6 Шаг 4: Адаптивный медианный фильтр . . . 30
4.7 Шаг 5: ZUPT с гистерезисом . . . 30
4.8 Шаг 6: Трапецеидальное интегрирование (резервный режим) . . . 31
4.9 Шаг 7: Комплементарный фильтр (резервный режим) . . . 32
4.10 Теоретическое обоснование: слияние датчиков . . . 32
4.10.1 Комплементарный фильтр как частный случай фильтра Калмана 32
4.10.2 Частотная интерпретация . . . 32
4.10.3 Трапецеидальное vs. Эйлерово интегрирование . . . 33
4.10.4 Дискретная модель пространства состояний . . . 33
4.11 Сводная схема конвейера обработки . . . 33

5 Pure Pursuit: алгоритм возврата домой **34**
5.1 Постановка задачи . . . 34
5.2 Запись путевых точек . . . 34
5.3 Алгоритм Pure Pursuit . . . 35

5.3.1 Вычисление угла на цель . . . 35
5.3.2 Пропорциональный угловой регулятор . . . 35
5.3.3 Косинусное масштабирование линейной скорости . . . 36
5.3.4 Финальный подход . . . 36
5.4 Нормализация угла . . . 36
5.5 Геометрия алгоритма Pure Pursuit . . . 37
5.6 Теоретическое обоснование: замкнутая система управления . . . 37
5.6.1 Структура контура управления . . . 37
5.6.2 Нелинейное масштабирование и устойчивость . . . 37
5.6.3 Связь с кинематической моделью . . . 38

6 SLAM: вероятностная карта занятости на log-odds 39
6.1 Вероятностная модель карты . . . 39
6.1.1 Log-odds представление . . . 39
6.2 Трассировка лучей (Ray Tracing) . . . 40
6.3 Ультразвуковой конус . . . 41
6.4 Преобразование координат . . . 41
6.5 Кластеризация объектов с Евклидовой метрикой . . . 42
6.6 Параметры сетки . . . 42

7 Одометрия: кинематика дифференциального привода 43
7.1 Модель дифференциального привода . . . 43
7.2 Микшер дифференциального привода . . . 43
7.3 Интегрирование одометрии (кинематические уравнения) . . . 44
7.4 Кватернионное кодирование угла курса . . . 45
7.5 Сводная диаграмма . . . 45
7.6 Теоретическое обоснование: кинематика и динамика . . . 45
7.6.1 Неголономная модель робота . . . 45
7.6.2 Модель unicycle в пространстве состояний . . . 46
7.6.3 Связь с моделью двигателя . . . 46
7.6.4 Погрешность метода Эйлера . . . 46
7.7 Параметры . . . 46

8 Кватернионы, проекции и преобразования ориентации 47
8.1 Углы Эйлера из акселерометра (сырые) . . . 47
8.2 Кватернион → угол курса (yaw из ROS2 Odometry) . . . 48
8.2.1 Кватернионы и ориентация в 3D . . . 48
8.2.2 Формула извлечения угла курса . . . 48
8.3 Полярные координаты лазерного сканера → декартовы . . . 48
8.4 Геометрическое представление кватерниона . . . 49
8.5 Полная матрица вращения из кватерниона . . . 49

9 Калибровка IMU и цифровая фильтрация 50
9.1 Аппаратная часть: MPU6050 . . . 50
9.1.1 Масштабные коэффициенты . . . 50
9.2 Цифровой фильтр нижних частот (DLPF) . . . 51
9.3 Экспоненциальное скользящее среднее (EMA) . . . 52
9.4 Калибровка при старте . . . 53
9.4.1 Смещение нуля гироскопа . . . 53

9.4.2 Вектор гравитации в системе координат тела . . . 53
9.4.3 Начальная ориентация (home) . . . 53
9.5 Конвейер обработки IMU . . . 54
9.6 Сводная таблица параметров . . . 54
9.7 C++ реализация (оптимизированная) . . . 54
9.8 Теоретическое обоснование: цифровая фильтрация . . . 55
9.8.1 EMA как дискретный ФНЧ . . . 55
9.8.2 Частотная характеристика EMA . . . 55
9.8.3 Связь с аналоговым RC-фильтром . . . 55
9.8.4 DLPF как аппаратный ФНЧ . . . 56

10 A* Pathfinding и алгоритм Брезенхэма 57
10.1 Постановка задачи . . . 57
10.2 Построение сетки занятости . . . 58
10.3 Преобразование координат . . . 58
10.4 Алгоритм A* . . . 59
10.4.1 Оценочная функция A* . . . 59
10.4.2 Октильная эвристика . . . 59
10.4.3 Стоимость переходов . . . 60
10.5 Алгоритм Брезенхэма (Line-of-Sight) . . . 60
10.6 Сглаживание пути (Line-of-Sight Pruning) . . . 61
10.7 Геометрия A* на сетке . . . 61
10.8 Сводная таблица параметров . . . 61

11 Физический симулятор: кинематика, сенсоры, камера 62
11.1 Кинематика робота (SimRobot) . . . 62
11.1.1 Дифференциальные уравнения кинематики . . . 63
11.1.2 Нормализация угла . . . 63
11.1.3 Коллизия со стенами . . . 63
11.2 Ультразвуковой сенсор (SimSensors) . . . 64
11.2.1 Пересечение луч–плоскость (стены) . . . 64
11.2.2 Пересечение луч–сфера (шарики) . . . 65
11.2.3 Шумовая модель . . . 65
11.3 Симуляция IMU . . . 65
11.4 Модель перспективной камеры (SimDetector) . . . 66
11.4.1 Проверка видимости (Field of View) . . . 66
11.4.2 Горизонтальная проекция на экран . . . 66
11.4.3 Модель pinhole-камеры (размер объекта) . . . 66
11.4.4 Вертикальная позиция (глубинная подсказка) . . . 67
11.4.5 Модель доверия (confidence) . . . 67
11.5 Параметры симулятора . . . 67

12 Следование за человеком: оценка дистанции и P-регулятор 68
12.1 Постановка задачи . . . 68
12.2 Оценка расстояния по bounding box . . . 68
12.3 Рулевое управление (P-регулятор по углу) . . . 69
12.3.1 Нормализованная ошибка центрирования . . . 69
12.3.2 Угловая скорость (P-регулятор) . . . 69
12.3.3 Линейная скорость (P-регулятор с мёртвой зоной) . . . 69

12.4 Теоретическое обоснование: замкнутая система слежения 70
12.4.1 Двухконтурная структура 70
12.4.2 Передаточные функции контуров 70
12.4.3 Мёртвая зона и устойчивость 70
12.5 Таймаут потери цели 70
12.6 Сводная таблица параметров 71

13 Патрулирование: навигация по точкам и кватернионы 72
13.1 Постановка задачи 72
13.2 Проверка достижения точки 72
13.3 Формирование цели Nav2 (кватернион из yaw) 73
13.4 Геометрия патрулирования 73
13.5 Параметры 74

14 Геозона, преобразование сенсоров и реактивная навигация 75
14.1 Геозона: алгоритм Point-in-Polygon 75
14.1.1 Алгоритм трассировки луча (Ray Casting) 75
14.1.2 Построение карты занятости 76
14.2 Ультразвук → LaserScan: синтетическое сканирование 76
14.2.1 Геометрия конуса ультразвука 77
14.3 Реактивная навигация (fallback) 77
14.3.1 Пропорциональное масштабирование скорости 78
14.3.2 Конечный автомат реактивной навигации 78
14.3.3 Теоретическое обоснование: конечный автомат как дискретная система 78
14.4 Сводная таблица параметров 79

15 ШИМ-управление моторами и PCA9685 80
15.1 Контроллер PCA9685 80
15.1.1 Вычисление прескейлера 80
15.2 Slow-Decay режим управления мотором 81
15.2.1 Формулы Slow-Decay 81
15.2.2 Схема каналов моторов 81
15.3 Дифференциальный микшер 82
15.4 Теоретическое обоснование: электропривод 82
15.4.1 Модель двигателя постоянного тока 82
15.4.2 Slow-Decay и эффективный момент 82
15.4.3 Замкнутый контур «микшер + двигатель» 83
15.5 Параметры 83

Введение

Данный документ является техническим справочником по всем математическим, физическим и дифференциальным системам, реализованным в проекте **SAMURAI** — автономного наземного робота на гусеничном шасси.

Документ построен **модульно**: каждая глава соответствует отдельному программному модулю и содержит:

- Ссылку на исходный файл с указанием строк кода
- Математическое описание алгоритма
- Точный фрагмент кода с пояснениями
- Физический смысл каждого уравнения

Структура проекта (математические компоненты):

Файл	**Язык**	**Содержание**
pi_nodes/filters/velocity_ekf.py	Python	2D Velocity EKF
pi_nodes/filters/ekf_imu.py	Python	IMU yaw 1D Kalman
pi_nodes/filters/accel_position.py	Python	Слияние датчиков
pi_nodes/nodes/path_recorder_node.py	Python	Pure Pursuit
pi_nodes/nodes/slam_map_node.py	Python	SLAM log-odds
pi_nodes/nodes/imu_node.py	Python	IMU калибровка + EMA
pi_nodes/nodes/fallback_nav_node.py	Python	Реактивная навигация
compute_node/simulator.py	Python	Физика + A* + Bresenham
compute_node/dashboard_node.py	Python	Кватернионы
compute_node/geofence_map_publisher.py	Python	Point-in-Polygon
compute_node/depth_to_scan_node.py	Python	УЗ → LaserScan
ros_ws/.../follow_me_node.py	Python	Pinhole-камера + P-рег.
ros_ws/.../patrol_node.py	Python	Навигация + кватернионы
ros_ws/.../motor_node.cpp	C++	Одометрия + ШИМ-микшер
ros_ws/.../imu_node.cpp	C++	I2C IMU (оптимизация)

Chapter 1

Теоретические основы систем управления

Данная глава содержит математический аппарат теории автоматического управления (ТАУ), на котором базируются все алгоритмы проекта SAMURAI.

1.1 Модель в пространстве состояний

Любой объект управления может быть описан системой дифференциальных уравнений первого порядка — **уравнениями состояния**:

$$\boxed{\dot{\boldsymbol{x}}(t) = \mathbf{A}\,\boldsymbol{x}(t) + \mathbf{B}\,\boldsymbol{u}(t), \qquad \boldsymbol{y}(t) = \mathbf{C}\,\boldsymbol{x}(t) + \mathbf{D}\,\boldsymbol{u}(t)} \tag{1.1}$$

где:

- $\boldsymbol{x}(t) = (x_1, x_2, \ldots, x_n)^\mathsf{T} \in \mathbb{R}^n$ — **вектор состояния**
- $\boldsymbol{u}(t) = (u_1, \ldots, u_r)^\mathsf{T} \in \mathbb{R}^r$ — вектор управляющих воздействий
- $\boldsymbol{y}(t) = (y_1, \ldots, y_m)^\mathsf{T} \in \mathbb{R}^m$ — вектор выходных переменных
- $\mathbf{A} \in \mathbb{R}^{n\times n}$ — матрица системы (динамики)
- $\mathbf{B} \in \mathbb{R}^{n\times r}$ — матрица управления
- $\mathbf{C} \in \mathbb{R}^{m\times n}$ — матрица наблюдения
- $\mathbf{D} \in \mathbb{R}^{m\times r}$ — матрица прямого прохода

Примечание

Структурная схема модели в пространстве состояний содержит интегратор ($\int$) для перехода от $\dot{\boldsymbol{x}}$ к $\boldsymbol{x}$, блоки умножения на матрицы $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, $\mathbf{D}$ и сумматоры.

1.1.1 Применение в проекте SAMURAI

Каждый фильтр Калмана в проекте является реализацией модели в пространстве состояний:

Модуль	Вектор состояния $\boldsymbol{x}$	Размерность n
Velocity EKF (гл. 2)	$(v_x, v_y)^\mathsf{T}$	2
IMU EKF (гл. 3)	$(\psi, b_z)^\mathsf{T}$	2
AccelPosition (гл. 4)	$(x, y, v_x, v_y)^\mathsf{T}$	4
Одометрия (гл. 7)	$(x, y, \theta)^\mathsf{T}$	3

1.2 Линеаризация нелинейных систем

Реальные объекты, включая наземного робота, описываются **нелинейными** уравнениями:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}), \qquad \boldsymbol{y}(t) = \boldsymbol{g}(\boldsymbol{x}, \boldsymbol{u}) \tag{1.2}$$

1.2.1 Пример: транспортная тележка

Модель движения мобильного робота (транспортной тележки) описывается системой шести дифференциальных уравнений:

$$\dot{\theta} = -\frac{1}{T_1}\theta + \frac{1}{T_1}u_1 \tag{1.3}$$

$$\dot{f} = -\frac{1}{T_2}f + \frac{1}{T_2}u_2 \tag{1.4}$$

$$\dot{v} = -\frac{\alpha}{m}v + \frac{1}{m}f \tag{1.5}$$

$$\dot{x} = v\cos(\psi + \theta) \tag{1.6}$$

$$\dot{\psi} = \frac{1}{L}v\sin(\theta) \tag{1.7}$$

$$\dot{z} = -v\sin(\psi + \theta) \tag{1.8}$$

где θ — угол поворота колёс, ψ — угол рыскания, f — движущая сила, v — скорость, x, z — координаты; T_1, T_2, α, m, L — параметры объекта.

Уравнения (1.6)–(1.8) **нелинейны** (содержат sin и cos), что делает систему нелинейной.

Примечание

Модель робота SAMURAI использует ту же структуру: уравнения одометрии (гл. 7) $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$ нелинейны из-за тригонометрических функций от угла курса θ.

1.2.2 Метод линеаризации по Тейлору (матрица Якоби)

Линеаризация проводится в окрестности **опорного режима** $(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}})$, в котором $\boldsymbol{f}(\bar{\boldsymbol{x}}, \bar{\boldsymbol{u}}) = \boldsymbol{0}$.

Разложение в ряд Тейлора с отбрасыванием членов высших порядков:

$$\boxed{\Delta\dot{\boldsymbol{x}} = \mathbf{A}\,\Delta\boldsymbol{x} + \mathbf{B}\,\Delta\boldsymbol{u}, \qquad \Delta\boldsymbol{y} = \mathbf{C}\,\Delta\boldsymbol{x} + \mathbf{D}\,\Delta\boldsymbol{u}} \tag{1.9}$$

где матрицы — **якобианы**, вычисленные в опорной точке:

$$\mathbf{A} = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}, \quad \mathbf{B} = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}, \quad \mathbf{C} = \left.\frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}}, \quad \mathbf{D} = \left.\frac{\partial \boldsymbol{g}}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}},\bar{\boldsymbol{u}}} \tag{1.10}$$

1.2.3 Пример: якобиан для транспортной тележки

Для опорного режима: $v(t) = v^0$, $\theta(t) = 0$, $\psi(t) = 0$:

$$\mathbf{A} = \begin{pmatrix} -1/T_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1/T_2 & 0 & 0 & 0 & 0 \\ 0 & 1/m & -\alpha/m & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ v^0/L & 0 & 0 & 0 & 0 & 0 \\ -v^0 & 0 & 0 & 0 & -v^0 & 0 \end{pmatrix} \tag{1.11}$$

1.2.4 Связь с EKF в проекте SAMURAI

Расширенный фильтр Калмана (EKF) — это именно **линеаризация на каждом шаге**. Матрица перехода $\mathbf{F}$ в фильтре Калмана (гл. 3, формула (3.6)) — это якобиан нелинейной модели:

$$\mathbf{F}_k = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}_{k-1}} = \begin{pmatrix} 1 & -\Delta t \\ 0 & 1 \end{pmatrix} \tag{1.12}$$

Здесь нелинейная модель $\psi_k = \psi_{k-1} + (g_z - b_z)\Delta t$ линеаризуется относительно текущего состояния $(\psi_{k-1}, b_{z,k-1})$, а якобиан $\mathbf{F}$ содержит производные $\partial\dot{\psi}/\partial\psi = 1$, $\partial\dot{\psi}/\partial b_z = -\Delta t$.

1.3 Передаточная функция

Для линейной стационарной системы (1.1) **передаточная функция** определяется через преобразование Лапласа:

$$\boxed{W(p) = \mathbf{C}(p\mathbf{E}_n - \mathbf{A})^{-1}\mathbf{B} + \mathbf{D}} \tag{1.13}$$

где $p = \frac{d}{dt}$ — оператор дифференцирования, $\mathbf{E}_n$ — единичная матрица.

1.3.1 Характеристический полином и резольвента

Резольвента матрицы $\mathbf{A}$:

$$(p\mathbf{E}_n - \mathbf{A})^{-1} = \frac{1}{\chi_n(p)}\widetilde{\mathbf{B}}(p) \tag{1.14}$$

где $\chi_n(p) = \det(p\mathbf{E}_n - \mathbf{A}) = p^n + a_1 p^{n-1} + \ldots + a_n$ — характеристический полином, а $\widetilde{\mathbf{B}}(p)$ — присоединённая матрица.

1.3.2 Алгоритм Фаддеева–Леверье

Коэффициенты характеристического полинома и матрицы $\widetilde{\mathbf{B}}_k$ вычисляются рекуррентно:

$$\boxed{\begin{array}{ll} \widetilde{\mathbf{B}}_1 = \mathbf{E}_n, & a_1 = -\mathrm{tr}(\mathbf{A}\widetilde{\mathbf{B}}_1) \\ \widetilde{\mathbf{B}}_k = \mathbf{A}\widetilde{\mathbf{B}}_{k-1} + a_{k-1}\mathbf{E}_n, & a_k = -\dfrac{1}{k}\mathrm{tr}(\mathbf{A}\widetilde{\mathbf{B}}_k) \\ & \mathrm{k} = 2, \ldots, \mathrm{n} \end{array}} \tag{1.15}$$

Проверка: $\mathbf{A}\widetilde{\mathbf{B}}_n + a_n\mathbf{E}_n = \mathbf{0}$ (теорема Гамильтона–Кели).

1.3.3 Уравнение «вход–выход»

Передаточная функция связывает выход y с входом u через дифференциальное уравнение:

$$p^n y + a_1 p^{n-1} y + \ldots + a_n y = b_0 p^n u + b_1 p^{n-1} u + \ldots + b_n u \tag{1.16}$$

Примечание

В проекте SAMURAI фильтры Калмана работают в **дискретном** пространстве состояний: $\boldsymbol{x}_{k+1} = \mathbf{F}\,\boldsymbol{x}_k + \mathbf{B}\,\boldsymbol{u}_k$, что является дискретным аналогом непрерывной модели (1.1). Переход к дискретной модели: $\mathbf{A}_d = e^{\mathbf{A}T}$, $\mathbf{B}_d = \int_0^T e^{\mathbf{A}\tau} d\tau\,\mathbf{B}$.

1.4 Устойчивость систем управления

1.4.1 Определение устойчивости

Система (1.1) **асимптотически устойчива**, если решение $\boldsymbol{x}(t) \to \mathbf{0}$ при $t \to \infty$ для любых начальных условий $\boldsymbol{x}(0)$.

1.4.2 Критерий собственных значений

Система устойчива тогда и только тогда, когда все собственные числа матрицы $\mathbf{A}$ имеют отрицательные вещественные части:

$$\boxed{\mathrm{Re}(\lambda_i) < 0, \quad \forall\, i = 1, \ldots, n} \tag{1.17}$$

где λ_i — корни характеристического уравнения $\det(\lambda\mathbf{E} - \mathbf{A}) = 0$.

1.4.3 Критерий Гурвица

Для характеристического полинома $\chi(p) = p^n + a_1 p^{n-1} + \ldots + a_n$ формируется матрица Гурвица:

$$\mathbf{H} = \begin{pmatrix} a_1 & a_3 & a_5 & \cdots \\ 1 & a_2 & a_4 & \cdots \\ 0 & a_1 & a_3 & \cdots \\ 0 & 1 & a_2 & \cdots \\ \vdots & & & \ddots \end{pmatrix} \tag{1.18}$$

Условие устойчивости: все угловые миноры матрицы $\mathbf{H}$ положительны:

$$\Delta_1 = a_1 > 0, \quad \Delta_2 = a_1 a_2 - a_3 > 0, \quad \ldots, \quad \Delta_n > 0 \tag{1.19}$$

1.4.4 Решение уравнения состояния

Аналитическое решение линейной системы (формула Коши):

$$\boldsymbol{x}(t) = e^{\mathbf{A}(t-t_0)}\boldsymbol{x}(t_0) + \int_{t_0}^{t} e^{\mathbf{A}(t-\tau)}\mathbf{B}\,\boldsymbol{u}(\tau)\,d\tau \tag{1.20}$$

где $e^{\mathbf{A}t}$ — матричная экспонента. Первое слагаемое — свободное движение (от начальных условий), второе — вынужденное движение (от входного воздействия).

1.4.5 Жорданова форма и матричная экспонента

Через замену $\boldsymbol{x} = \mathbf{S}\boldsymbol{z}$ система приводится к жордановой форме:

$$\dot{\boldsymbol{z}} = \mathbf{J}\boldsymbol{z} + \mathbf{S}^{-1}\mathbf{B}\boldsymbol{u}, \quad \mathbf{J} = \mathbf{S}^{-1}\mathbf{A}\mathbf{S} \tag{1.21}$$

Для диагональной жордановой формы экспонента:

$$e^{\mathbf{J}t} = \operatorname{diag}\bigl(e^{\lambda_1 t}, e^{\lambda_2 t}, \ldots, e^{\lambda_n t}\bigr) \tag{1.22}$$

Отсюда видно: при $\operatorname{Re}(\lambda_i) < 0$ каждая мода затухает экспоненциально, что обеспечивает устойчивость.

1.5 Дискретные системы и фильтр Калмана

1.5.1 Дискретная модель в пространстве состояний

$$\boldsymbol{x}_{k+1} = \mathbf{A}_d\,\boldsymbol{x}_k + \mathbf{B}_d\,\boldsymbol{u}_k, \qquad \boldsymbol{y}_k = \mathbf{C}_d\,\boldsymbol{x}_k + \mathbf{D}_d\,\boldsymbol{u}_k \tag{1.23}$$

где $\mathbf{A}_d = e^{\mathbf{A}T}$, T — период дискретизации.

1.5.2 Общие уравнения фильтра Калмана

Фильтр Калмана решает задачу оптимальной оценки состояния стохастической системы:

$$\boldsymbol{x}_k = \mathbf{F}\,\boldsymbol{x}_{k-1} + \mathbf{B}\,\boldsymbol{u}_{k-1} + \boldsymbol{w}_{k-1}, \qquad \boldsymbol{z}_k = \mathbf{H}\,\boldsymbol{x}_k + \boldsymbol{v}_k \tag{1.24}$$

где $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ — процессный шум, $\boldsymbol{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ — шум измерений.

Шаг предсказания (Predict):

$$\hat{\boldsymbol{x}}_k^- = \mathbf{F}\,\hat{\boldsymbol{x}}_{k-1} + \mathbf{B}\,\boldsymbol{u}_{k-1} \tag{1.25}$$

$$\mathbf{P}_k^- = \mathbf{F}\,\mathbf{P}_{k-1}\,\mathbf{F}^\mathsf{T} + \mathbf{Q} \tag{1.26}$$

Шаг коррекции (Update):

$$\boldsymbol{y}_k = \boldsymbol{z}_k - \mathbf{H}\,\hat{\boldsymbol{x}}_k^- \qquad \text{(инновация)} \tag{1.27}$$

$$\mathbf{S}_k = \mathbf{H}\,\mathbf{P}_k^-\,\mathbf{H}^\mathsf{T} + \mathbf{R} \qquad \text{(ковариация инновации)} \tag{1.28}$$

$$\mathbf{K}_k = \mathbf{P}_k^-\,\mathbf{H}^\mathsf{T}\,\mathbf{S}_k^{-1} \qquad \text{(усиление Калмана)} \tag{1.29}$$

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + \mathbf{K}_k\,\boldsymbol{y}_k \qquad \text{(обновление состояния)} \tag{1.30}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\,\mathbf{P}_k^- \qquad \text{(обновление ковариации)} \tag{1.31}$$

Формула

Фильтр Калмана **минимизирует** дисперсию ошибки оценки $E[\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2]$. Усиление $\mathbf{K}$ автоматически взвешивает доверие между предсказанием (модель) и измерением (датчик):

- $\mathbf{R} \ll \mathbf{P}^-$: $\mathbf{K} \to \mathbf{H}^{-1}$ — доверяем датчику
- $\mathbf{R} \gg \mathbf{P}^-$: $\mathbf{K} \to \mathbf{0}$ — доверяем модели

1.5.3 Расширенный фильтр Калмана (EKF)

Для нелинейной системы $\boldsymbol{x}_k = \boldsymbol{f}(\boldsymbol{x}_{k-1}, \boldsymbol{u}_{k-1})$, $\boldsymbol{z}_k = \boldsymbol{h}(\boldsymbol{x}_k)$ используется **линеаризация на каждом шаге**:

$$\mathbf{F}_k = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\hat{\boldsymbol{x}}_{k-1}}, \qquad \mathbf{H}_k = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\hat{\boldsymbol{x}}_k^-} \tag{1.32}$$

Затем якобианы $\mathbf{F}_k$, $\mathbf{H}_k$ подставляются в стандартные уравнения фильтра Калмана (1.26)–(1.31).

1.6 Модель электродвигателя постоянного тока

Двигатели робота SAMURAI управляются ШИМ-сигналами через драйвер TB6612. Электромеханические процессы в двигателе постоянного тока описываются дифференциальным уравнением второго порядка:

$$\boxed{T_a^2\ddot{\omega} + T_y\dot{\omega} + \omega = k\,u} \tag{1.33}$$

где:

- ω — угловая скорость вала двигателя
- u — управляющее воздействие (напряжение / ШИМ)
- T_a — электромеханическая постоянная времени
- T_y — электрическая постоянная времени
- k — коэффициент передачи

1.6.1 Пространство состояний для двигателя

Вводя $x_1 = \omega$, $x_2 = \dot{\omega}$:

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1/T_a^2 & -T_y/T_a^2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} 0 \\ k/T_a^2 \end{pmatrix} u \tag{1.34}$$

Характеристический полином:

$$\chi(\lambda) = \lambda^2 + \frac{T_y}{T_a^2}\lambda + \frac{1}{T_a^2} \tag{1.35}$$

Условие устойчивости (по Гурвицу): $T_y > 0$, $T_a > 0$ — выполняется для всех реальных двигателей.

1.6.2 Передаточная функция двигателя

$$W(p) = \frac{k}{T_a^2 p^2 + T_y p + 1} \tag{1.36}$$

При $T_a \ll T_y$ (типично для малых моторов) двигатель приближённо описывается звеном первого порядка:

$$W(p) \approx \frac{k}{T_y p + 1} \tag{1.37}$$

Примечание

В проекте SAMURAI ШИМ-управление (гл. 15) работает с нормированным throttle $t \in [-1, 1]$, что соответствует $u/u_{\max}$. Драйвер TB6612 в режиме Slow Decay обеспечивает линейную зависимость $\omega \propto t$ в установившемся режиме.

1.7 Пропорциональный регулятор (P-регулятор)

1.7.1 Закон управления

$$u(t) = K_P \cdot e(t), \qquad e(t) = r(t) - y(t) \tag{1.38}$$

где $r(t)$ — задание, $y(t)$ — выход, K_P — коэффициент усиления.

1.7.2 Передаточная функция замкнутой системы

Для объекта с передаточной функцией $W_{\text{obj}}(p)$ и P-регулятора с коэффициентом K_P:

$$W_{\text{зс}}(p) = \frac{K_P \cdot W_{\text{obj}}(p)}{1 + K_P \cdot W_{\text{obj}}(p)} \tag{1.39}$$

1.7.3 Статическая ошибка

P-регулятор не может полностью устранить установившуюся ошибку при ступенчатом воздействии:

$$e_{\text{уст}} = \frac{1}{1 + K_P \cdot k_{\text{obj}}} \tag{1.40}$$

Увеличение K_P снижает ошибку, но может привести к потере устойчивости.

1.7.4 Применение P-регуляторов в проекте

Модуль	**Управляемая переменная**	K_P	**Ограничение**
Pure Pursuit (гл. 5)	Угловая скорость ω	2.5	$\lvert\omega\rvert \leq 0.8$
Follow-Me (гл. 12)	Угловая скорость ω	0.8	—
Follow-Me (гл. 12)	Линейная скорость v	0.3	$v \leq 0.20$
Patrol (гл. 13)	Целевая позиция	—	$d < 0.20\,\text{м}$

1.8 Переходная характеристика

Переходная характеристика $h(t)$ — реакция системы на единичное ступенчатое воздействие $u(t) = 1(t)$ при нулевых начальных условиях:

$$h(t) = \mathbf{C} \int_0^t e^{\mathbf{A}\tau}\, d\tau\, \mathbf{B} + \mathbf{D} \tag{1.41}$$

При отсутствии нулевых собственных значений у $\mathbf{A}$:

$$h(t) = \mathbf{C}\left(e^{\mathbf{A}t} - \mathbf{E}\right)\mathbf{A}^{-1}\mathbf{B} + \mathbf{D} \tag{1.42}$$

Для системы второго порядка с ω_n (собственная частота) и ζ (коэффициент демпфирования):

$$h(t) = 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1-\zeta^2}} \sin\left(\omega_n \sqrt{1-\zeta^2}\, t + \arccos\zeta\right) \tag{1.43}$$

Примечание

Косинусное масштабирование скорости в Pure Pursuit (гл. 5, формула (5.5)) обеспечивает плавную переходную характеристику без перерегулирования при развороте робота.

1.9 Управляемость и наблюдаемость

1.9.1 Матрица управляемости

$$\mathbf{S}_v = \left[\mathbf{B},\ \mathbf{AB},\ \mathbf{A}^2\mathbf{B},\ \ldots,\ \mathbf{A}^{n-1}\mathbf{B}\right] \tag{1.44}$$

Система полностью управляема $\Leftrightarrow \operatorname{rank}(\mathbf{S}_v) = n$.

1.9.2 Матрица наблюдаемости

$$\mathbf{S}_o = \begin{pmatrix} \mathbf{C} \\ \mathbf{CA} \\ \vdots \\ \mathbf{CA}^{n-1} \end{pmatrix} \tag{1.45}$$

Система полностью наблюдаема $\Leftrightarrow \operatorname{rank}(\mathbf{S}_o) = n$.

Примечание

Фильтр Калмана может оценивать состояние только **наблюдаемой** системы. EKF для угла курса (гл. 3) наблюдаем, так как гироскоп измеряет $\omega_z = \dot{\psi} - b_z$, что позволяет оценить и ψ, и b_z.

1.10 Сводная таблица: теория → реализация

Теоретический концепт	Глава	Реализация
$\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x} + \mathbf{B}\boldsymbol{u}$	2, 3	Predict-шаг EKF
Якобиан $\mathbf{F} = \partial\boldsymbol{f}/\partial\boldsymbol{x}$	3	Матрица перехода $\mathbf{F}$
$\mathbf{K} = \mathbf{PH}^\top\mathbf{S}^{-1}$	2	Усиление Калмана
$u = K_P \cdot e$	5, 12	Угловой P-регулятор
$T_a^2\ddot{\omega} + T_y\dot{\omega} = ku$	15	ШИМ-управление мотором
$\operatorname{Re}(\lambda_i) < 0$	3	Устойчивость EKF
$e^{\mathbf{A}t}$	4	Интегрирование позиции
$\boldsymbol{x}_{k+1} = \mathbf{A}_d\boldsymbol{x}_k$	7	Дискретная одометрия

Chapter 2

2D Velocity Extended Kalman Filter

Файл исходного кода

pi_nodes/filters/velocity_ekf.py (251 строка, Python)

2.1 Постановка задачи

Для точного позиционирования гусеничного робота необходимо объединить два источника скорости:

- **Колёсная одометрия** — стабильна, не имеет дрейфа, но не учитывает проскальзывание гусениц на неровном покрытии.
- **Акселерометр (однократное интегрирование)** — улавливает реальное скольжение, но накапливает ошибку интегрирования (дрейф).

Оба источника объединяются **фильтром Калмана**, который автоматически взвешивает вклад каждого по его статистическим шумовым характеристикам.

2.2 Вектор состояния и модель

2.2.1 Вектор состояния

Состояние фильтра — скорость робота в *мировой* системе координат:

$$\boldsymbol{x} = \begin{pmatrix} v_x \\ v_y \end{pmatrix} \in \mathbb{R}^2 \tag{2.1}$$

Позиция получается последовательным интегрированием фильтрованной скорости.

2.2.2 Матрица ковариации

Поскольку numpy не используется, матрица 2×2 хранится в четырёх скалярах:

$$\mathbf{P} = \begin{pmatrix} p_{00} & p_{01} \\ p_{10} & p_{11} \end{pmatrix} \tag{2.2}$$

Начальное значение: $\mathbf{P} = 0.01 \cdot \mathbf{I}$.

2.3 Шаг предсказания (Prediction)

velocity_ekf.py, строки 90–120 — predict_wheel()

```
# Wheel-predicted velocity in world frame
vx_wheel = vx_cmd * cos(theta)
vy_wheel = vx_cmd * sin(theta)

# Prediction: state transitions toward wheel command
alpha = 0.5
self.vx = (1 - alpha) * self.vx + alpha * vx_wheel
self.vy = (1 - alpha) * self.vy + alpha * vy_wheel

# Process noise: covariance grows
self._p00 += self.q_wheel * dt
self._p11 += self.q_wheel * dt

# Integrate position using current fused velocity
self.x += self.vx * dt
self.y += self.vy * dt
```

Математика предсказания:

Команда колёс задаёт вперёднаправленную скорость v_{cmd} в теле робота. Поворот на угол курса θ (из IMU) переводит её в мировую систему координат:

$$\begin{pmatrix} v_x^{\text{wheel}} \\ v_y^{\text{wheel}} \end{pmatrix} = v_{\text{cmd}} \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix} \tag{2.3}$$

Смешивание с текущим состоянием ($\alpha = 0.5$):

$$\boldsymbol{x}^- = (1-\alpha)\,\boldsymbol{x} + \alpha\,\boldsymbol{x}^{\text{wheel}} \tag{2.4}$$

Рост ковариации (процессный шум $q_{\text{wheel}} = 0.01$):

$$p_{00}^- = p_{00} + q_{\text{wheel}} \cdot \Delta t, \quad p_{11}^- = p_{11} + q_{\text{wheel}} \cdot \Delta t \tag{2.5}$$

Интегрирование позиции (правило Эйлера):

$$\boxed{x \mathrel{+}= v_x \cdot \Delta t, \quad y \mathrel{+}= v_y \cdot \Delta t} \tag{2.6}$$

2.4 Шаг коррекции (Update) по акселерометру

velocity_ekf.py, строки 122–173 — update_accel()

```
135 # Integrate accel to get velocity measurement
136 self._accel_vx += la_wx * dt
137 self._accel_vy += la_wy * dt
138
139 # Decay accel velocity to prevent unbounded drift
140 self._accel_vx *= 0.98
141 self._accel_vy *= 0.98
142
143 # Innovation: y = z - H*x  (H = I)
144 yx = self._accel_vx - self.vx
145 yy = self._accel_vy - self.vy
146
147 # S = P + R (innovation covariance)
148 r = self.r_accel + self.q_accel * dt
149 s00 = self._p00 + r
150 s11 = self._p11 + r
151
152 # Kalman gain: K = P * S^-1
153 k00 = self._p00 / s00
154 k11 = self._p11 / s11
155
156 # State update
157 self.vx += k00 * yx
158 self.vy += k11 * yy
159
160 # Covariance update: P = (I - K*H) * P
161 self._p00 *= (1 - k00)
162 self._p11 *= (1 - k11)
163 self._p01 *= (1 - 0.5 * (k00 + k11))
164 self._p10 *= (1 - 0.5 * (k00 + k11))
```

Математика коррекции:

Однократное интегрирование ускорения (измерение скорости):

$$\boldsymbol{v}_k^{\text{accel}} = \boldsymbol{v}_{k-1}^{\text{accel}} + \boldsymbol{a}^{\text{world}} \cdot \Delta t \tag{2.7}$$

Экспоненциальное затухание для борьбы с дрейфом:

$$\boldsymbol{v}^{\text{accel}} \leftarrow 0.98 \cdot \boldsymbol{v}^{\text{accel}} \tag{2.8}$$

Инновация (невязка):

$$\boldsymbol{y} = \boldsymbol{z} - \mathbf{H}\,\boldsymbol{x}^{-}, \quad \mathbf{H} = \mathbf{I} \quad \Rightarrow \quad \boldsymbol{y} = \boldsymbol{v}^{\text{accel}} - \boldsymbol{v}^{\text{state}} \tag{2.9}$$

Ковариация инновации:

$$\mathbf{S} = \mathbf{P}^- + \mathbf{R}, \quad R = r_{\text{accel}} + q_{\text{accel}} \cdot \Delta t \tag{2.10}$$

Усиление Калмана (для диагональной $\mathbf{S}$, т.е. поэлементно):

$$\boxed{K_{ii} = \frac{p_{ii}^-}{S_{ii}}, \quad i \in \{0, 1\}} \tag{2.11}$$

Обновление состояния:

$$\boldsymbol{x} = \boldsymbol{x}^- + \mathbf{K}\,\boldsymbol{y} \tag{2.12}$$

Обновление ковариации (формула Джозефа в диагональном виде):

$$\boxed{p_{ii} \leftarrow (1 - K_{ii})\, p_{ii}} \tag{2.13}$$

2.5 ZUPT — обновление нулевой скорости

velocity_ekf.py, строки 181–207 — update_zupt()

```
# Kalman gain (measurement = zero, R = r_zupt)
s00 = self._p00 + r
s11 = self._p11 + r
k00 = self._p00 / s00
k11 = self._p11 / s11

# Update: z = [0, 0], innovation = -[vx, vy]
self.vx -= k00 * self.vx
self.vy -= k11 * self.vy

# Covariance update
self._p00 *= (1 - k00)
self._p11 *= (1 - k11)
```

Когда робот стоит, измерение $\boldsymbol{z} = \mathbf{0}$ с очень малым шумом $r_{\text{zupt}} = 10^{-4}$. Инновация: $\boldsymbol{y} = \mathbf{0} - \boldsymbol{v} = -\boldsymbol{v}$. Это приводит к быстрому обнулению скорости через стандартный шаг Калмана.

2.6 Неголономное ограничение

velocity_ekf.py, строки 209–228 — apply_nonholonomic()

```
213 cy, sy = cos(theta), sin(theta)
214 fwd = self.vx * cy + self.vy * sy     # forward component
215 lat = -self.vx * sy + self.vy * cy    # lateral component
216
217 lat *= self._lateral_decay            # kill 70% of lateral vel
218
219 self.vx = fwd * cy - lat * sy
220 self.vy = fwd * sy + lat * cy
```

Гусеничный робот **не может двигаться боком**. После шага Калмана производится проекция скорости на направление движения:

$$v_{\text{fwd}} = v_x \cos\theta + v_y \sin\theta \tag{2.14}$$

$$v_{\text{lat}} = -v_x \sin\theta + v_y \cos\theta \tag{2.15}$$

Боковая составляющая подавляется: $v_{\text{lat}} \leftarrow 0.3 \cdot v_{\text{lat}}$.

Обратное преобразование в мировую СК (поворот матрицей $\mathbf{R}^{-1} = \mathbf{R}^{\mathsf{T}}$):

$$\begin{pmatrix} v_x \\ v_y \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} v_{\text{fwd}} \\ v_{\text{lat}} \end{pmatrix} \tag{2.16}$$

2.7 Модуль скорости

velocity_ekf.py, строки 243–245

```
243 @property
244 def velocity_magnitude(self) -> float:
245     return sqrt(self.vx * self.vx + self.vy * self.vy)
```

$$|\boldsymbol{v}| = \sqrt{v_x^2 + v_y^2} \tag{2.17}$$

2.8 Теоретическое обоснование: связь с ТАУ

2.8.1 Модель в пространстве состояний

Velocity EKF реализует дискретную модель в пространстве состояний (см. гл. 1, формула (1.1)):

$$\underbrace{\begin{pmatrix} v_x \\ v_y \end{pmatrix}_k}_{\boldsymbol{x}_k} = \underbrace{\begin{pmatrix} 1-\alpha & 0 \\ 0 & 1-\alpha \end{pmatrix}}_{\mathbf{F}} \underbrace{\begin{pmatrix} v_x \\ v_y \end{pmatrix}_{k-1}}_{\boldsymbol{x}_{k-1}} + \underbrace{\begin{pmatrix} \alpha\cos\theta & 0 \\ 0 & \alpha\sin\theta \end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix} v_{\text{cmd}} \\ v_{\text{cmd}} \end{pmatrix}}_{\boldsymbol{u}} \tag{2.18}$$

2.8.2 Устойчивость фильтра

Собственные числа матрицы перехода $\mathbf{F}$: $\lambda_{1,2} = 1 - \alpha = 0.5$. Поскольку $|\lambda_i| < 1$ для дискретной системы, фильтр **асимптотически устойчив** (ошибка сходится к нулю).

Это дискретный аналог критерия устойчивости (1.17): для дискретных систем все собственные числа должны лежать внутри единичного круга $|\lambda_i| < 1$.

2.8.3 Оптимальность усиления Калмана

Усиление Калмана $K_{ii} = p_{ii}/(p_{ii}+R)$ минимизирует апостериорную дисперсию оценки:

$$\frac{\partial}{\partial K} E\big[(x-\hat{x})^2\big] = 0 \quad \Rightarrow \quad K^* = \frac{P^-}{P^- + R} \tag{2.19}$$

При $R \to 0$ (идеальный датчик): $K \to 1$, фильтр доверяет измерению. При $R \to \infty$ (шумный датчик): $K \to 0$, фильтр доверяет модели.

2.9 Сводная таблица параметров

Параметр	**Символ**	**Значение**	**Физический смысл**
q_wheel	q_w	$0.01\,(m/s)^2$	Шум процесса (колёса)
q_accel	q_a	$0.05\,(m/s)^2$	Скорость дрейфа акселерометра
r_accel	r_a	$0.1\,(m/s)^2$	Шум измерения акселерометра
r_zupt	r_z	$10^{-4}\,(m/s)^2$	Доверие к нулевой скорости
lateral_decay	—	0.3	Подавление бокового движения

Chapter 3

IMU EKF: одномерный фильтр Калмана для угла курса

Файл исходного кода

pi_nodes/filters/ekf_imu.py (171 строка, Python)

3.1 Постановка задачи

Наземный робот движется по плоской поверхности. Нас интересуют три угла Эйлера:

- **Курс (yaw)** ψ — угол поворота вокруг вертикальной оси; главный навигационный параметр.
- **Крен (roll)** ϕ и **тангаж (pitch)** θ — информационные, для контроля наклона на склоне.

Подход:

- **Курс**: 1D-фильтр Калмана по интегрированию гироскопа g_z с отслеживанием дрейфа нуля.
- **Крен/тангаж**: прямой расчёт через atan2 из акселерометра, без фильтрации (для наземного робота достаточно).

3.2 Вектор состояния

$$\boldsymbol{x}_{\text{yaw}} = \begin{pmatrix} \psi \\ b_z \end{pmatrix} \tag{3.1}$$

где ψ — угол курса (рад), b_z — дрейф нуля гироскопа по оси Z (рад/с).

Матрица ковариации 2×2 (без numpy, три скаляра):

$$\mathbf{P} = \begin{pmatrix} P_\psi & P_\times \\ P_\times & P_b \end{pmatrix} \tag{3.2}$$

3.3 Шаг предсказания: интегрирование гироскопа

ekf_imu.py, строки 85–122 — predict()

```
97  # Bias-corrected gz
98  wz = gz - self._gz_bias
99
100 # ZUPT: if angular rate is tiny, skip integration
101 if abs(wz) < self._zupt_threshold:   # 0.01 rad/s
102     self._P_yaw  += self.q_angle * dt * 0.1
103     self._P_bias += self.q_bias * dt
104     return
105
106 # Yaw prediction: simple integration
107 self._yaw += wz * dt
108
109 # Normalize to [-pi, pi]
110 self._yaw = (self._yaw + pi) % (2 * pi) - pi
111
112 # Covariance prediction: F = [[1, -dt], [0, 1]]
113 # P = F @ P @ F.T + Q  (manual 2x2)
114 p11 = self._P_yaw
115 p12 = self._P_cross
116 p22 = self._P_bias
117
118 self._P_yaw   = p11 + (-dt)*p12 + (-dt)*(p12 + (-dt)*p22) \
119                 + self.q_angle * dt
120 self._P_cross = p12 + (-dt) * p22
121 self._P_bias  = p22 + self.q_bias * dt
```

Математика предсказания:

Скорректированная угловая скорость:

$$\omega_z = g_z - b_z \tag{3.3}$$

ZUPT (Zero-velocity Update): при $|\omega_z| < 0.01$ рад/с интегрирование пропускается, ковариация растёт медленнее (коэффициент 0.1).

Интегрирование угла курса:

$$\psi_k = \psi_{k-1} + \omega_z \cdot \Delta t \tag{3.4}$$

Нормализация в $[-\pi, \pi]$:

$$\psi \leftarrow (\psi + \pi) \bmod 2\pi - \pi \tag{3.5}$$

Матрица перехода системы:

$$\mathbf{F} = \begin{pmatrix} 1 & -\Delta t \\ 0 & 1 \end{pmatrix} \tag{3.6}$$

Уравнение предсказания ковариации:

$$\boxed{\mathbf{P}^- = \mathbf{F}\,\mathbf{P}\,\mathbf{F}^\mathsf{T} + \mathbf{Q}} \tag{3.7}$$

где матрица процессного шума:

$$\mathbf{Q} = \begin{pmatrix} q_{\text{angle}} \cdot \Delta t & 0 \\ 0 & q_{\text{bias}} \cdot \Delta t \end{pmatrix} \tag{3.8}$$

В развёрнутом виде (ручное умножение 2×2):

$$P_\psi^- = P_\psi - \Delta t \cdot P_\times - \Delta t(P_\times - \Delta t \cdot P_b) + q_{\text{angle}} \cdot \Delta t \tag{3.9}$$
$$P_\times^- = P_\times - \Delta t \cdot P_b \tag{3.10}$$
$$P_b^- = P_b + q_{\text{bias}} \cdot \Delta t \tag{3.11}$$

3.4 Шаг коррекции: крен и тангаж из акселерометра

ekf_imu.py, строки 124–147 — update()

```
132 a_mag = sqrt(ax*ax + ay*ay + az*az)
133 if abs(a_mag - self.g) > self.accel_gate * self.g:
134     return   # robot is accelerating -- skip gravity update
135
136 # Raw roll/pitch from gravity vector
137 raw_roll  = atan2(ay, az)
138 raw_pitch = atan2(-ax, sqrt(ay*ay + az*az))
139
140 # Subtract home position -> angles relative to startup
141 self._roll  = raw_roll  - self._home_roll
142 self._pitch = raw_pitch - self._home_pitch
143
144 # Normalize to [-pi, pi]
145 self._roll  = (self._roll  + pi) % (2*pi) - pi
146 self._pitch = (self._pitch + pi) % (2*pi) - pi
```

Математика определения крена и тангажа:

Когда робот не ускоряется, вектор акселерометра совпадает с вектором гравитации $\boldsymbol{g} = (0, 0, -g)$ в мировой СК:

$$|\boldsymbol{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2} \approx g = 9.81\,\text{м/с}^2 \tag{3.12}$$

Условие пропуска обновления: $\big||\boldsymbol{a}| - g\big| > 0.3g$.

Крен (вращение вокруг оси X):

$$\boxed{\phi_{\text{raw}} = \text{atan2}(a_y,\, a_z)} \tag{3.13}$$

Тангаж (вращение вокруг оси Y):

$$\boxed{\theta_{\text{raw}} = \operatorname{atan2}\left(-a_x,\ \sqrt{a_y^2 + a_z^2}\right)} \tag{3.14}$$

Относительный угол (от положения при старте):

$$\phi = \phi_{\text{raw}} - \phi_{\text{home}}, \quad \theta = \theta_{\text{raw}} - \theta_{\text{home}} \tag{3.15}$$

Примечание

Акселерометр **не** корректирует угол курса ψ: для этого нужен магнитометр. Поэтому угол курса накапливает ошибку интегрирования — именно поэтому используется фильтр Калмана с отслеживанием дрейфа b_z.

3.5 Теоретическое обоснование: линеаризация и устойчивость

3.5.1 Нелинейная модель и якобиан

Нелинейная модель гироскопа с дрейфом нуля:

$$\boldsymbol{f}(\boldsymbol{x}, u) = \begin{pmatrix} \psi + (g_z - b_z)\Delta t \\ b_z \end{pmatrix} \tag{3.16}$$

Якобиан этой функции по вектору состояния $\boldsymbol{x} = (\psi, b_z)^\mathsf{T}$ (метод линеаризации, гл. 1, раздел 1.2.2):

$$\mathbf{F} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \begin{pmatrix} \partial f_1/\partial\psi & \partial f_1/\partial b_z \\ \partial f_2/\partial\psi & \partial f_2/\partial b_z \end{pmatrix} = \begin{pmatrix} 1 & -\Delta t \\ 0 & 1 \end{pmatrix} \tag{3.17}$$

3.5.2 Устойчивость фильтра

Собственные числа $\mathbf{F}$: $\lambda_{1,2} = 1$ (двукратный корень). Без коррекции система **нейтрально устойчива** — ошибка не растёт, но и не убывает. Именно шаг коррекции (Update) через усиление Калмана обеспечивает асимптотическую устойчивость, уменьшая ковариацию $\mathbf{P}$.

3.5.3 Наблюдаемость

Матрица наблюдаемости для $\mathbf{H} = (1, 0)$ (прямое измерение ψ через внешний датчик):

$$\mathbf{S}_o = \begin{pmatrix} \mathbf{H} \\ \mathbf{HF} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1 & -\Delta t \end{pmatrix}, \quad \operatorname{rank}(\mathbf{S}_o) = 2 \tag{3.18}$$

Ранг равен размерности состояния $\Rightarrow$ система **полностью наблюдаема**: дрейф b_z может быть оценён из измерений угла ψ.

3.5.4 Связь с передаточной функцией

В операторной форме модель гироскопа:

$$\psi(p) = \frac{1}{p}\big(g_z(p) - b_z(p)\big) \tag{3.19}$$

Это интегрирующее звено с передаточной функцией $W(p) = 1/p$, что объясняет накопление ошибки (дрейф) при отсутствии коррекции.

3.6 Сводная таблица параметров

Параметр	**Символ**	**Значение**	**Смысл**
q_angle	q_ψ	0.001	Шум процесса по курсу
q_bias	q_b	0.0001	Шум дрейфа нуля гиро
r_accel	r_a	0.5	Шум измерения акселерометра
accel_gate	—	0.3	Порог ворот акселерометра (30%)
zupt_threshold	—	0.01 рад/с	Порог ZUPT ($\approx$0.6 °/с)

Chapter 4

Оценка позиции по акселерометру с слиянием датчиков

Файл исходного кода

pi_nodes/filters/accel_position.py (420 строк, Python)

4.1 Архитектура системы слияния датчиков

Модуль объединяет три источника информации:

1. **Акселерометр** — мгновенное линейное ускорение (50 Гц)
2. **Колёсная одометрия** — команды моторов (20 Гц)
3. **Orientation from IMU** — углы Эйлера для поворота фреймов

4.2 Константы физики

accel_position.py, строки 44–69

```
44 EARTH_OMEGA = 7.2921e-5    # rad/s -- Earth angular velocity
45 DEFAULT_LATITUDE = 55.75   # Moscow
46
47 ZUPT_GYRO_ENTER  = 0.015   # rad/s
48 ZUPT_ACCEL_ENTER = 0.10    # m/s^2
49 ZUPT_GYRO_EXIT   = 0.04    # rad/s
50 ZUPT_ACCEL_EXIT  = 0.25    # m/s^2
51 ZUPT_ENTER_COUNT = 5       # 100ms @ 50Hz
52 ZUPT_EXIT_COUNT  = 3       # 60ms  @ 50Hz
53
54 LATERAL_DECAY = 0.5        # kill 50% lateral velocity per step
```

Угловая скорость Земли (для компенсации Кориолиса):

$$\Omega = 7.2921 \times 10^{-5}\,\text{рад/с} \tag{4.1}$$

Проекция на широту (Москва, $\varphi = 55.75°$):

$$\omega_z = \Omega \sin \varphi \tag{4.2}$$

4.3 Шаг 1: Вычитание гравитации

accel_position.py, строки 211–225 — update_imu()

```
211 # Remove gravity -- DIRECT SUBTRACTION
212 la_bx = ax - self._g_body[0]
213 la_by = ay - self._g_body[1]
214 la_bz = az - self._g_body[2]
215
216 # Delta orientation correction (slopes only)
217 d_roll  = roll
218 d_pitch = pitch
219 if abs(d_roll) > 0.01 or abs(d_pitch) > 0.01:
220     g = self._g_mag
221     delta_gx = -g * sin(d_pitch)
222     delta_gy  =  g * sin(d_roll)
223     la_bx -= delta_gx
224     la_by -= delta_gy
```

Прямое вычитание вектора гравитации, откалиброванного при старте:

$$\boldsymbol{a}_{\text{body}}^{\text{lin}} = \boldsymbol{a}_{\text{meas}} - \boldsymbol{g}_{\text{body}} \tag{4.3}$$

Примечание

Почему прямое вычитание, а не ротационный метод?
IMU физически наклонён на шасси. В покое он показывает, например, $a_x = 0.707\,\text{м/с}^2$ — чистая гравитация через наклон. При вычитании: $0.707 - 0.707 = 0$ — нет ложного движения. Ротационный метод предполагает крен = тангаж = 0, давая $g_{\text{body}} = (0, 0, 9.81)$, и оставляет $0.707\,\text{м/с}^2$ как «движение».

Поправка на наклон (при нахождении на уклоне $> 0.6°$):

$$\Delta g_x = -g \sin \theta_{\text{pitch}}, \quad \Delta g_y = g \sin \phi_{\text{roll}} \tag{4.4}$$

4.4 Шаг 2: Поворот из тела в мировую СК

accel_position.py, строки 227–230

```
227 # Body -> World frame (rotate by yaw)
228 cy, sy = cos(yaw), sin(yaw)
229 la_wx = la_bx * cy - la_by * sy
230 la_wy = la_bx * sy + la_by * cy
```

Матрица поворота вокруг вертикальной оси (z-axis rotation):

$$\boxed{\begin{pmatrix} a_x^{\text{world}} \\ a_y^{\text{world}} \end{pmatrix} = \mathbf{R}(\psi) \begin{pmatrix} a_x^{\text{body}} \\ a_y^{\text{body}} \end{pmatrix}, \quad \mathbf{R}(\psi) = \begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix}} \tag{4.5}$$

4.5 Шаг 3: Компенсация эффекта Кориолиса

accel_position.py, строки 232–234

```
232 # Earth rotation compensation (Coriolis)
233 la_wx -= 2.0 * self._omega_z * self.vy
234 la_wy += 2.0 * self._omega_z * self.vx
```

Сила Кориолиса на единицу массы:

$$\boldsymbol{a}_{\text{Cor}} = -2\,\boldsymbol{\Omega} \times \boldsymbol{v} \tag{4.6}$$

В 2D (только вертикальная проекция $\omega_z = \Omega \sin\varphi$):

$$\begin{cases} a_{Cx} = -2\,\omega_z\,v_y \\ a_{Cy} = +2\,\omega_z\,v_x \end{cases} \tag{4.7}$$

Вычитание ускорения Кориолиса из измеренного:

$$a_x^{\text{lin}} \leftarrow a_x^{\text{lin}} - 2\omega_z v_y, \quad a_y^{\text{lin}} \leftarrow a_y^{\text{lin}} + 2\omega_z v_x \tag{4.8}$$

4.6 Шаг 4: Адаптивный медианный фильтр

accel_position.py, строки 236–250

```
236 # Median filter (reject vibration spikes)
237 self._buf_ax.append(la_wx)
238 self._buf_ay.append(la_wy)
239
240 if len(self._buf_ax) >= 3:
241     la_wx = _median_of_3(
242         self._buf_ax[-1], self._buf_ax[-2], self._buf_ax[-3])
243     la_wy = _median_of_3(
244         self._buf_ay[-1], self._buf_ay[-2], self._buf_ay[-3])
245
246 # Noise threshold -- zero out tiny accelerations
247 if abs(la_wx) < ACCEL_NOISE_THRESHOLD:    # 0.15 m/s^2
248     la_wx = 0.0
```

Медиана трёх последних значений (без сортировки — одно ветвление):

$$\hat{a} = \mathrm{median}(a_k,\, a_{k-1},\, a_{k-2}) \tag{4.9}$$

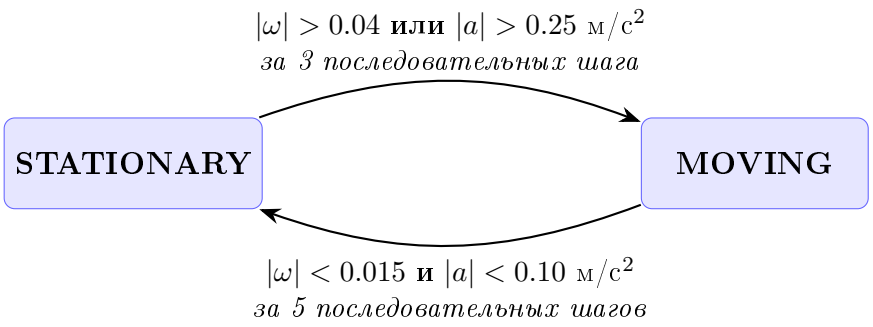
Пороговое обнуление шума: $|a| < 0.15\,\text{м/с}^2 \Rightarrow a = 0$.

4.7 Шаг 5: ZUPT с гистерезисом

accel_position.py, строки 252–305

```
252 gyro_mag  = sqrt(gx*gx + gy*gy + gz*gz)
253 accel_mag = sqrt(la_wx*la_wx + la_wy*la_wy)
254
255 if self._stationary:
256     # Need STRONG evidence of motion to exit
257     if (self._cmd_moving and
258         (gyro_mag > ZUPT_GYRO_EXIT or        # 0.04 rad/s
259          accel_mag > ZUPT_ACCEL_EXIT)):       # 0.25 m/s^2
260         self._exit_count += 1
261     if self._exit_count >= ZUPT_EXIT_COUNT:  # 3 samples
262         self._stationary = False
263 else:
264     # Easy to re-enter stationary
265     if (not self._cmd_moving or
266         (gyro_mag < ZUPT_GYRO_ENTER and      # 0.015 rad/s
267          accel_mag < ZUPT_ACCEL_ENTER)):     # 0.10 m/s^2
268         self._enter_count += 1
269     if self._enter_count >= ZUPT_ENTER_COUNT: # 5 samples
270         self._stationary = True
```

Двухпороговый автомат состояний (гистерезис):



Высокие пороги *выхода* ($> 0.04, > 0.25$) и низкие пороги *входа* ($< 0.015, < 0.10$) предотвращают ложные переходы на границе движения/покоя.

4.8 Шаг 6: Трапецеидальное интегрирование (резервный режим)

accel_position.py, строки 315–334 — fallback

```
315 # Trapezoidal integration (more accurate than Euler)
316 self.vx += 0.5 * (la_wx + self._prev_la_x) * dt
317 self.vy += 0.5 * (la_wy + self._prev_la_y) * dt
318
319 # Non-holonomic constraint
320 fwd = self.vx * cy + self.vy * sy
321 lat = -self.vx * sy + self.vy * cy
322 lat *= LATERAL_DECAY
323 self.vx = fwd * cy - lat * sy
324 self.vy = fwd * sy + lat * cy
325
326 # Position integration
327 self._accel_x += self.vx * dt
328 self._accel_y += self.vy * dt
```

Метод трапеций (порядок точности $O(\Delta t^2)$, лучше Эйлера $O(\Delta t)$):

$$v_k = v_{k-1} + \frac{\Delta t}{2}\left(a_k + a_{k-1}\right) \tag{4.10}$$

4.9 Шаг 7: Комплементарный фильтр (резервный режим)

accel_position.py, строки 376–386 — blend()

```
378 # Complementary filter: alpha*wheel + (1-alpha)*accel
379 self._alpha = 0.95 * self._alpha + 0.05 * self._alpha_moving
380 a = self._alpha
381 self.x = a * self._wheel_x + (1.0 - a) * self._accel_x
382 self.y = a * self._wheel_y + (1.0 - a) * self._accel_y
383
384 # Slow drift correction of accel toward wheel
385 self._accel_x = 0.995 * self._accel_x + 0.005 * self._wheel_x
386 self._accel_y = 0.995 * self._accel_y + 0.005 * self._wheel_y
```

Комплементарный фильтр с адаптивным α:

$$\boldsymbol{p} = \alpha\, \boldsymbol{p}_{\text{wheel}} + (1-\alpha)\, \boldsymbol{p}_{\text{accel}} \tag{4.11}$$

где $\alpha \in [0.95, 1.0]$ (при движении растёт вес одометрии). Медленная коррекция дрейфа акселерометра: $\boldsymbol{p}_{\text{accel}} \leftarrow 0.995\, \boldsymbol{p}_{\text{accel}} + 0.005\, \boldsymbol{p}_{\text{wheel}}$.

4.10 Теоретическое обоснование: слияние датчиков

4.10.1 Комплементарный фильтр как частный случай фильтра Калмана

Комплементарный фильтр (4.11) можно рассматривать как **стационарный фильтр Калмана** с фиксированным усилением:

$$\hat{x} = \alpha\, x_{\text{odom}} + (1-\alpha)\, x_{\text{accel}} = x_{\text{accel}} + \alpha\,(x_{\text{odom}} - x_{\text{accel}}) \tag{4.12}$$

Сравнивая с (1.30): $\hat{x} = \hat{x}^- + K \cdot y$, видим что α играет роль **усиления Калмана** K.

4.10.2 Частотная интерпретация

В частотной области комплементарный фильтр выделяет:

- **Низкие частоты** из одометрии (стабильная, но дрейфующая)
- **Высокие частоты** из акселерометра (шумный, но без дрейфа)

$$X(p) = \underbrace{\frac{\alpha}{1+\alpha/p}}_{\text{ФНЧ}} X_{\text{odom}}(p) + \underbrace{\frac{p/\alpha}{1+p/\alpha}}_{\text{ФВЧ}} X_{\text{accel}}(p) \tag{4.13}$$

Частота среза: $f_c = \alpha/(2\pi\Delta t) \approx 0.95/(2\pi \cdot 0.02) \approx 7.6\,\text{Гц}$.

4.10.3 Трапецеидальное vs. Эйлерово интегрирование

Метод	Формула	Погрешность
Эйлер вперёд	$v_k = v_{k-1} + a_k \Delta t$	$O(\Delta t)$
Трапеция	$v_k = v_{k-1} + \frac{a_k + a_{k-1}}{2} \Delta t$	$O(\Delta t^2)$

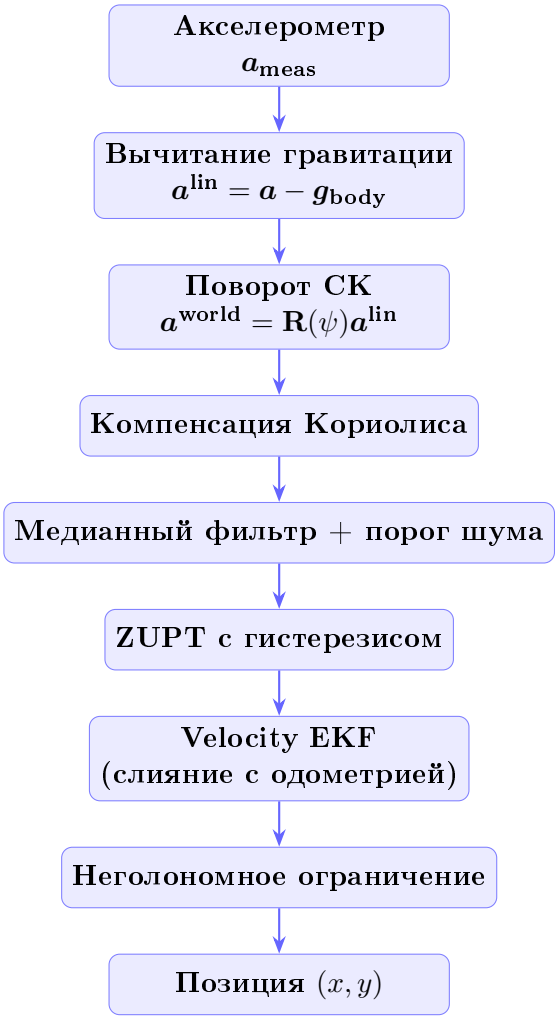
При частоте 50 Гц ($\Delta t = 0.02$ с) разница в погрешности составляет $\Delta t/2 = 0.01$, т.е. трапеция точнее Эйлера в ≈100 раз для гладких сигналов.

4.10.4 Дискретная модель пространства состояний

Полная система AccelPosition в форме пространства состояний (гл. 1, формула (1.23)):

$$\underbrace{\begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}_k}_{\boldsymbol{x}_k} = \underbrace{\begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\mathbf{F}} \boldsymbol{x}_{k-1} + \underbrace{\begin{pmatrix} \Delta t^2/2 & 0 \\ 0 & \Delta t^2/2 \\ \Delta t & 0 \\ 0 & \Delta t \end{pmatrix}}_{\mathbf{B}} \begin{pmatrix} a_x \\ a_y \end{pmatrix} \tag{4.14}$$

4.11 Сводная схема конвейера обработки



Chapter 5

Pure Pursuit: алгоритм возврата домой

Файл исходного кода

pi_nodes/nodes/path_recorder_node.py (342 строки, Python)

5.1 Постановка задачи

Модуль записывает путь робота в виде набора опорных точек и затем проводит робота обратно по тому же пути в обратном порядке (возврат домой).

- **Запись**: каждые 3 см или 4.5° поворота.
- **Воспроизведение**: алгоритм Pure Pursuit с финальным фазой точного наведения.

5.2 Запись путевых точек

path_recorder_node.py, строки 135–148 — _maybe_record_waypoint()

```
139 dx = self._x - self._last_record_x
140 dy = self._y - self._last_record_y
141 dist = math.sqrt(dx * dx + dy * dy)
142 dtheta = abs(self._angle_diff(self._theta, self._last_record_theta
    ))
143
144 if dist >= RECORD_INTERVAL_M or dtheta >= RECORD_INTERVAL_RAD:
145     self._path.append((self._x, self._y, self._theta))
146     # RECORD_INTERVAL_M = 0.03 m, RECORD_INTERVAL_RAD = 0.08 rad
```

Расстояние между точками (Евклидова метрика):

$$d = \sqrt{(x - x_{\text{last}})^2 + (y - y_{\text{last}})^2} \tag{5.1}$$

Запись происходит при: $d \geq 0.03$ м ИЛИ $|\Delta\theta| \geq 0.08$ рад.

5.3 Алгоритм Pure Pursuit

path_recorder_node.py, строки 206–243 — _control_loop()

```
206 # Pure pursuit: simultaneous linear + angular
207 bearing = math.atan2(dy, dx)
208 angle_error = self._angle_diff(bearing, self._theta)
209
210 if in_final_approach:
211     angular   = FINAL_ANGULAR_KP * angle_error    # Kp = 3.0
212     angular   = max(-REPLAY_ANGULAR_SPEED, min(
            REPLAY_ANGULAR_SPEED, angular))
213     cos_factor = max(0.0, math.cos(angle_error))
214     dist_ratio = dist / FINAL_APPROACH_RADIUS      # 0..1
215     linear     = FINAL_LINEAR_SPEED * cos_factor * dist_ratio
216     linear     = max(REPLAY_MIN_LINEAR * 0.6 * cos_factor, linear)
217     if abs(angle_error) > 0.8:
218         linear = 0.0
219 else:
220     angular    = REPLAY_ANGULAR_KP * angle_error  # Kp = 2.5
221     angular    = max(-REPLAY_ANGULAR_SPEED, min(
            REPLAY_ANGULAR_SPEED, angular))
222     cos_factor = max(0.0, math.cos(angle_error))
223     dist_factor = min(1.0, dist / 0.15)
224     linear     = REPLAY_LINEAR_SPEED * cos_factor * dist_factor
225     linear     = max(REPLAY_MIN_LINEAR * cos_factor, linear)
226     if abs(angle_error) > 1.2:
227         linear = 0.0
```

5.3.1 Вычисление угла на цель

Пеленг (bearing) на текущую целевую точку (t_x, t_y):

$$\beta = \operatorname{atan2}(t_y - y,\ t_x - x) \tag{5.2}$$

Угловая ошибка (кратчайший путь в $[-\pi, \pi]$):

$$e_\theta = \text{angle_diff}(\beta, \psi) \in [-\pi, \pi] \tag{5.3}$$

5.3.2 Пропорциональный угловой регулятор

$$\omega = K_P \cdot e_\theta, \quad \omega \in [-\omega_{\max}, +\omega_{\max}] \tag{5.4}$$

Режим	K_P	$\omega_{\max}$
Обычный Pure Pursuit	2.5	0.8 рад/с
Финальный подход	3.0	0.8 рад/с

5.3.3 Косинусное масштабирование линейной скорости

Ключевая идея Pure Pursuit: при большой угловой ошибке снижать линейную скорость через косинус:

$$\cos\text{-factor} = \max\big(0,\, \cos(e_\theta)\big) \tag{5.5}$$

$$v = v_{\max} \cdot \cos\text{-factor} \cdot d_{\text{factor}} \tag{5.6}$$

Коэффициент дистанции:

$$d_{\text{factor}} = \min\left(1,\, \frac{d}{0.15\,\text{м}}\right) \tag{5.7}$$

При $|e_\theta| > \pi/2$: $\cos(e_\theta) \leq 0 \Rightarrow v = 0$ (только поворот на месте).

5.3.4 Финальный подход

В радиусе 0.12 м от конечной точки:

$$v = v_{\text{final}} \cdot \cos(e_\theta) \cdot \frac{d}{r_{\text{final}}} \tag{5.8}$$

Скорость пропорциональна оставшемуся расстоянию — плавное торможение.

Параметр	Значение	Описание
REPLAY_LINEAR_SPEED	0.20 м/с	Скорость воспроизведения
FINAL_LINEAR_SPEED	0.08 м/с	Скорость финального подхода
REPLAY_LOOKAHEAD_M	0.10 м	Дистанция look-ahead
REPLAY_HOME_TOLERANCE	0.015 м	Допуск финальной точки
REPLAY_GOAL_TOLERANCE	0.04 м	Допуск промежуточных точек

5.4 Нормализация угла

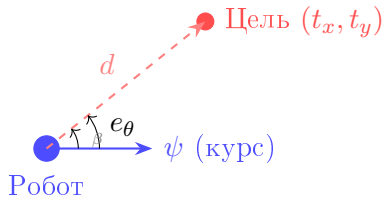
path_recorder_node.py, строки 312–320 — _angle_diff()

```
313 @staticmethod
314 def _angle_diff(target, current):
315     """Shortest signed angle difference, in [-pi, pi]."""
316     d = target - current
317     while d > math.pi:
318         d -= 2.0 * math.pi
319     while d < -math.pi:
320         d += 2.0 * math.pi
321     return d
```

$$\Delta\theta = \big[(\beta - \psi + \pi) \bmod 2\pi\big] - \pi \tag{5.9}$$

Это гарантирует, что угловая ошибка всегда в диапазоне $[-\pi, \pi]$, а поворот всегда происходит по кратчайшему пути.

5.5 Геометрия алгоритма Pure Pursuit



$$\beta = \operatorname{atan2}(t_y - y, t_x - x), \quad e_\theta = \beta - \psi$$

5.6 Теоретическое обоснование: замкнутая система управления

5.6.1 Структура контура управления

Алгоритм Pure Pursuit реализует **замкнутый P-регулятор** (гл. 1, раздел 1.7) для угловой скорости:

$$\omega = K_P \cdot e_\theta, \quad e_\theta = \beta - \psi \tag{5.10}$$

Передаточная функция замкнутой системы «регулятор + робот» (робот как интегратор $\dot{\psi} = \omega$):

$$W_{\text{зс}}(p) = \frac{K_P/p}{1 + K_P/p} = \frac{K_P}{p + K_P} \tag{5.11}$$

Полюс замкнутой системы: $\lambda = -K_P = -2.5$, что даёт экспоненциальную сходимость $e_\theta(t) = e_\theta(0)\, e^{-2.5t}$ с постоянной времени $\tau = 1/K_P = 0.4\,\text{с}$.

5.6.2 Нелинейное масштабирование и устойчивость

Косинусное масштабирование $v = v_{\max}\cos(e_\theta)$ создаёт **нелинейную связь** между угловой ошибкой и линейной скоростью. Это эквивалентно функции Ляпунова:

$$V(e_\theta) = 1 - \cos(e_\theta) \geq 0 \tag{5.12}$$

При $|e_\theta| < \pi/2$: $\dot{V} = \sin(e_\theta) \cdot \dot{e}_\theta < 0$ (при правильном знаке K_P), что гарантирует **асимптотическую устойчивость** по Ляпунову.

5.6.3 Связь с кинематической моделью

Полная нелинейная модель робота с Pure Pursuit (модель unicycle из гл. 7):

$$\begin{cases} \dot{x} = v\cos\theta \cdot \cos(e_\theta) \cdot d_{\text{factor}} \\ \dot{y} = v\sin\theta \cdot \cos(e_\theta) \cdot d_{\text{factor}} \\ \dot{\theta} = K_P \cdot e_\theta \end{cases} \tag{5.13}$$

Это нелинейная система вида $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ (гл. 1, формула (1.2)), линеаризация которой в окрестности $e_\theta = 0$ даёт устойчивую линейную систему с полюсом $\lambda = -K_P$.

Chapter 6

SLAM: вероятностная карта занятости на log-odds

Файл исходного кода

pi_nodes/nodes/slam_map_node.py (Python)

6.1 Вероятностная модель карты

Карта занятости представляет каждую ячейку сетки как бинарную случайную величину: свободно (F) или занято (O).

6.1.1 Log-odds представление

Вместо хранения вероятности $P(\mathtt{O})$ используется **логарифм отношения правдоподобий** (log-odds), устойчивый к крайним значениям 0 и 1:

$$l = \log \frac{P(\mathtt{O})}{1 - P(\mathtt{O})} = \log \frac{P(\mathtt{O})}{P(\mathtt{F})} \tag{6.1}$$

Обратное преобразование:

$$P(\mathtt{O}) = \frac{e^l}{1 + e^l} = \sigma(l) \tag{6.2}$$

Преимущества: обновление — простое сложение; нет умножения вероятностей.

slam_map_node.py, строки 42–48 — константы log-odds

```
42 L_OCC   =  0.85   # log-odds increment for occupied cell
43 L_FREE  = -0.40   # log-odds increment for free cell
44 L_MIN   = -2.0    # minimum log-odds (very confident free)
45 L_MAX   =  3.5    # maximum log-odds (very confident occupied)
46 L_THRESH_OCC  =  0.6   # above this -> occupied (output)
47 L_THRESH_FREE = -0.5   # below this -> free (output)
```

Физический смысл значений log-odds:

log-odds l	$P(0)$	Интерпретация
-2.0	11.9%	Очень уверенно свободно
-0.5	37.8%	Скорее свободно
0.0	50.0%	Неизвестно
$+0.6$	64.6%	Скорее занято
$+3.5$	97.1%	Очень уверенно занято

6.2 Трассировка лучей (Ray Tracing)

slam_map_node.py, строки 256–285 — _trace_ray()

```
256 def _trace_ray(self, ox, oy, angle, range_m):
257     cos_a = math.cos(angle)
258     sin_a = math.sin(angle)
259
260     step = CELL_SIZE_M * 0.7   # 3.5 cm step
261     dist = 0.0
262     hit_range = min(range_m, SENSOR_MAX_RANGE)
263
264     # Mark cells along ray as FREE
265     while dist < hit_range - OBSTACLE_THICKNESS:
266         wx = ox + cos_a * dist
267         wy = oy + sin_a * dist
268         ci, cj = self._world_to_cell(wx, wy)
269         if 0 <= ci < MAP_CELLS and 0 <= cj < MAP_CELLS:
270             idx = cj * MAP_CELLS + ci
271             self._grid[idx] = max(L_MIN, self._grid[idx] + L_FREE)
272         dist += step
273
274     # Mark obstacle cells as OCCUPIED
275     if range_m < SENSOR_MAX_RANGE - 0.05:
276         for d in range(3):
277             dd = hit_range + d * CELL_SIZE_M * 0.5
278             wx = ox + cos_a * dd
279             wy = oy + sin_a * dd
280             ci, cj = self._world_to_cell(wx, wy)
281             if 0 <= ci < MAP_CELLS and 0 <= cj < MAP_CELLS:
282                 idx = cj * MAP_CELLS + ci
283                 self._grid[idx] = min(L_MAX, self._grid[idx] +
                        L_OCC)
```

Параметрическое уравнение луча:

$$\begin{pmatrix} w_x(t) \\ w_y(t) \end{pmatrix} = \begin{pmatrix} o_x \\ o_y \end{pmatrix} + t \begin{pmatrix} \cos\alpha \\ \sin\alpha \end{pmatrix}, \quad t \in [0, r] \tag{6.3}$$

Байесовское обновление (вычитание для свободных, прибавление для занятых):

$$\boxed{l_{\text{new}} = \text{clamp}(l + \Delta l,\ L_{\min},\ L_{\max})} \tag{6.4}$$

где:

$$\Delta l = \begin{cases} L_{\text{FREE}} = -0.40 & \text{луч прошёл через ячейку} \\ L_{\text{OCC}} = +0.85 & \text{луч достиг препятствия} \end{cases}$$

6.3 Ультразвуковой конус

slam_map_node.py, строки 247–254 — сканирование конуса

```
247 # Ultrasonic cone: trace multiple rays within the FOV
248 angle_start = robot_theta - SENSOR_FOV_RAD   # theta - 15 deg
249 angle_end   = robot_theta + SENSOR_FOV_RAD   # theta + 15 deg
250
251 angle = angle_start
252 while angle <= angle_end:
253     self._trace_ray(robot_x, robot_y, angle, range_m)
254     angle += ANGULAR_RESOLUTION              # step = 0.05 rad ~ 3
            deg
```

Количество лучей на одно измерение:

$$N_{\text{rays}} = \left\lfloor \frac{2 \cdot \text{SENSOR_FOV_RAD}}{\text{ANGULAR_RESOLUTION}} \right\rfloor = \left\lfloor \frac{2 \times 0.26}{0.05} \right\rfloor = 10 \text{ лучей} \tag{6.5}$$

6.4 Преобразование координат

slam_map_node.py, строки 288–298 — world ↔ grid

```
288 def _world_to_cell(self, wx, wy):
289     ci = int((wx - self._origin_x) / CELL_SIZE_M)  # column
290     cj = int((wy - self._origin_y) / CELL_SIZE_M)  # row
291     return ci, cj
292
293 def _cell_to_world(self, ci, cj):
294     wx = self._origin_x + (ci + 0.5) * CELL_SIZE_M
295     wy = self._origin_y + (cj + 0.5) * CELL_SIZE_M
296     return wx, wy
```

Мировые координаты → ячейка сетки:

$$c_i = \left\lfloor \frac{w_x - o_x}{\delta} \right\rfloor, \quad c_j = \left\lfloor \frac{w_y - o_y}{\delta} \right\rfloor \tag{6.6}$$

Ячейка → центр в мировых координатах:

$$w_x = o_x + (c_i + 0.5)\,\delta, \quad w_y = o_y + (c_j + 0.5)\,\delta \tag{6.7}$$

где $o_x = o_y = -5$ м (начало координат карты), $\delta = 0.05$ м/ячейку.

6.5 Кластеризация объектов с Евклидовой метрикой

slam_map_node.py, строки 170–188 — _detections_cb()

```
172 # Search for nearest existing object (same class+colour)
173 for obj_id, obj in self._objects.items():
174     dx = obj['x'] - wx
175     dy = obj['y'] - wy
176     if math.sqrt(dx*dx + dy*dy) < OBJ_CLUSTER_RADIUS_M:  # 0.30 m
177         matched_id = obj_id
178         break
179
180 # Update position: sliding average
181 if matched_id:
182     alpha = 0.3
183     obj['x'] = round(alpha * wx + (1 - alpha) * obj['x'], 3)
184     obj['y'] = round(alpha * wy + (1 - alpha) * obj['y'], 3)
```

Проверка принадлежности к кластеру:

$$d_{\text{Euclid}} = \sqrt{(x_{\text{new}} - x_{\text{obj}})^2 + (y_{\text{new}} - y_{\text{obj}})^2} < 0.30\,\text{м} \tag{6.8}$$

Скользящее среднее позиции (EMA с $\alpha = 0.3$):

$$\boldsymbol{p}_{\text{obj}} \leftarrow 0.3\,\boldsymbol{p}_{\text{new}} + 0.7\,\boldsymbol{p}_{\text{obj}} \tag{6.9}$$

6.6 Параметры сетки

Параметр	Значение	Описание
MAP_SIZE_M	10 м	Размер карты (±5 м)
CELL_SIZE_M	0.05 м	Размер ячейки (5 см)
MAP_CELLS	200 × 200	Размер сетки
SENSOR_FOV_RAD	0.26 рад	Полу-угол конуса (≈15°)
SENSOR_MAX_RANGE	2.0 м	Макс. дальность HC-SR04

Chapter 7

Одометрия: кинематика дифференциального привода

Файлы исходного кода

ros_ws/src/robot_pkg_cpp/src/motor_node.cpp (C++, строки 244–302)
pi_nodes/nodes/motor_node.py (Python)

7.1 Модель дифференциального привода

Гусеничный робот управляется двумя независимыми бортами (левый/правый). Линейная и угловая скорости центра масс:

$$v_{\text{linear}} \in [-v_{\max}, v_{\max}], \quad \omega_{\text{angular}} \in [-\omega_{\max}, \omega_{\max}] \tag{7.1}$$

7.2 Микшер дифференциального привода

motor_node.cpp, строки 251–266 — controlLoop()

```
251 // Convert m/s -> throttle in [-1, 1]
252 double lin_t = (max_lin_ > 0.0) ? (linear_  / max_lin_) : 0.0;
253 double ang_t = (max_ang_ > 0.0) ? (angular_ / max_ang_) : 0.0;
254
255 // Differential-drive mixer
256 double left  = lin_t + ang_t;
257 double right = lin_t - ang_t;
258
259 // Normalise: keep ratio, clamp to [-1, 1]
260 double max_val = std::max({std::abs(left), std::abs(right), 1.0});
261 if (max_val > 1.0) { left /= max_val; right /= max_val; }
```

Формулы микшера (нормированные команды $\in [-1, 1]$):

$$\text{left} = \ell_{\text{lin}} + \ell_{\text{ang}}, \quad \text{right} = \ell_{\text{lin}} - \ell_{\text{ang}} \tag{7.2}$$

Нормировка для сохранения соотношения:

$$m = \max\bigl(1,\ |\text{left}|,\ |\text{right}|\bigr), \quad \text{left} \leftarrow \frac{\text{left}}{m}, \quad \text{right} \leftarrow \frac{\text{right}}{m} \tag{7.3}$$

7.3 Интегрирование одометрии (кинематические уравнения

motor_node.cpp, строки 268–273

```
268 // Open-loop odometry integration
269 x_     += linear_   * std::cos(theta_) * dt;
270 y_     += linear_   * std::sin(theta_) * dt;
271 theta_ += angular_  * dt;
```

Уравнения кинематики одноколейного робота (дискретная интеграция методом Эйлера вперёд):

$$x_{k+1} = x_k + v\cos(\theta_k) \cdot \Delta t \tag{7.4}$$
$$y_{k+1} = y_k + v\sin(\theta_k) \cdot \Delta t \tag{7.5}$$
$$\theta_{k+1} = \theta_k + \omega \cdot \Delta t \tag{7.6}$$

Это модель **unicycle** (одноколейная модель): скорость v направлена вдоль текущего курса θ.

Примечание

Открытоконтурная одометрия: модель не учитывает проскальзывание гусениц. Именно для компенсации этой ошибки и используется Velocity EKF (глава 2), который добавляет коррекцию от акселерометра.

7.4 Кватернионное кодирование угла курса

motor_node.cpp, строки 276–291 — publishOdom()

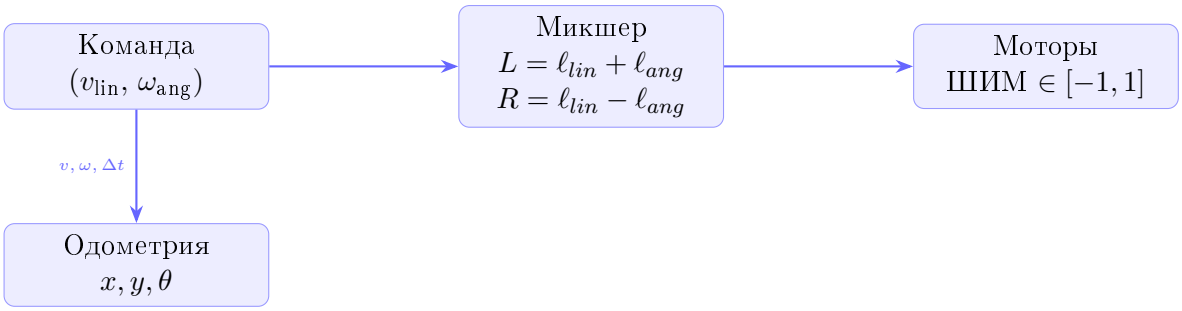
```
276 void publishOdom(const rclcpp::Time & stamp)
277 {
278   double sin_h = std::sin(theta_ / 2.0);
279   double cos_h = std::cos(theta_ / 2.0);
280
281   nav_msgs::msg::Odometry odom;
282   odom.pose.pose.orientation.z = sin_h;
283   odom.pose.pose.orientation.w = cos_h;
284   odom.twist.twist.linear.x    = linear_;
285   odom.twist.twist.angular.z   = angular_;
286   odom_pub_->publish(odom);
287 }
```

Поворот в 3D вокруг вертикальной оси Z кодируется кватернионом:

$$\boldsymbol{q} = \Big(\underbrace{0}_{q_w\text{'s missing part}}, \underbrace{0}_{q_x}, \underbrace{0}_{q_y}, \underbrace{\sin(\theta/2)}_{q_z}, \underbrace{\cos(\theta/2)}_{q_w} \Big) \tag{7.7}$$

Для чистого вращения вокруг Z: $q_x = q_y = 0$, поэтому достаточно хранить только q_z и q_w (что делает ROS2 сообщение Odometry).

7.5 Сводная диаграмма



7.6 Теоретическое обоснование: кинематика и динамика

7.6.1 Неголономная модель робота

Гусеничный робот подчиняется **неголономному ограничению** — не может двигаться боком. В терминах ТАУ это выражается как:

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0 \tag{7.8}$$

Это ограничение не интегрируется в голономную связь вида $f(x, y, \theta) = 0$, что делает систему **недоуправляемой** — размерность конфигурационного пространства $(x, y, \theta) = 3$ больше числа управлений $(v, \omega) = 2$.

7.6.2 Модель unicycle в пространстве состояний

Кинематическая модель (7.4)–(7.6) — это **нелинейная** система $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$:

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v \\ \omega \end{pmatrix} \tag{7.9}$$

Якобиан по состоянию (для EKF):

$$\mathbf{F} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \begin{pmatrix} 1 & 0 & -v\sin\theta\,\Delta t \\ 0 & 1 & v\cos\theta\,\Delta t \\ 0 & 0 & 1 \end{pmatrix} \tag{7.10}$$

7.6.3 Связь с моделью двигателя

Каждый борт робота приводится двигателем постоянного тока (гл. 1, раздел 1.6). Полная динамическая модель включает электромеханику:

$$T_a^2 \ddot{\omega}_{\text{motor}} + T_y \dot{\omega}_{\text{motor}} + \omega_{\text{motor}} = k\, u_{\text{PWM}} \tag{7.11}$$

Однако в проекте используется **кинематическая модель** (без учёта инерции), что оправдано малым отношением $T_a/T_{\text{control}} \ll 1$ (электрическая постоянная времени много меньше периода управления 50 мс).

7.6.4 Погрешность метода Эйлера

Интегрирование методом Эйлера вперёд имеет погрешность $O(\Delta t^2)$ на шаг и $O(\Delta t)$ глобально. Для периода $\Delta t = 0.05$ с и $v_{\max} = 0.5$ м/с максимальная ошибка позиции за шаг:

$$\epsilon \leq \frac{1}{2}|\ddot{x}|\Delta t^2 \approx \frac{v\omega\Delta t^2}{2} \leq \frac{0.5 \cdot 2.0 \cdot 0.0025}{2} = 1.25\,\text{мм} \tag{7.12}$$

Эта ошибка компенсируется Velocity EKF (гл. 2).

7.7 Параметры

Параметр	Значение	Описание
max_lin_	0.5 м/с	Максимальная линейная скорость
max_ang_	2.0 рад/с	Максимальная угловая скорость
Частота	20 Гц	Период управляющего цикла ($\Delta t = 0.05$ с)

Chapter 8

Кватернионы, проекции и преобразования ориентации

Файл исходного кода

compute_node/dashboard_node.py (Python, строки 298–521)

8.1 Углы Эйлера из акселерометра (сырые)

dashboard_node.py, строки 307–308 — _mqtt_imu()

```
306 # Raw pitch/roll from accel (no yaw)
307 pitch = math.degrees(math.atan2(ay, math.sqrt(ax*ax + az*az)))
308 roll  = math.degrees(math.atan2(-ax, az))
```

Тангаж (наклон вперёд/назад):

$$\theta_{\text{pitch}} = \operatorname{atan2}\Big(a_y,\ \sqrt{a_x^2 + a_z^2}\Big) \tag{8.1}$$

Крен (наклон влево/вправо):

$$\phi_{\text{roll}} = \operatorname{atan2}(-a_x,\ a_z) \tag{8.2}$$

Примечание

Функция $\operatorname{atan2}(y, x)$ возвращает угол в $(-\pi, \pi]$ и корректно обрабатывает все четыре квадранта, в отличие от $\arctan(y/x)$.

8.2 Кватернион → угол курса (yaw из ROS2 Odometry)

dashboard_node.py, строки 503–506 — _odom_cb()

```
503 q = msg.pose.pose.orientation
504 siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
505 cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z)
506 yaw = math.atan2(siny_cosp, cosy_cosp)
```

8.2.1 Кватернионы и ориентация в 3D

Кватернион единичной длины $\boldsymbol{q} = (q_w, q_x, q_y, q_z)$, $|\boldsymbol{q}| = 1$, представляет вращение в 3D:

$$\boldsymbol{q} = \cos\frac{\alpha}{2} + \hat{\boldsymbol{n}}\sin\frac{\alpha}{2} \tag{8.3}$$

где $\hat{\boldsymbol{n}}$ — ось вращения, α — угол.

8.2.2 Формула извлечения угла курса

Из матрицы вращения, построенной из кватерниона, угол вращения вокруг Z:

$$R_{32} = 2(q_w q_z + q_x q_y), \quad R_{33} = 1 - 2(q_y^2 + q_z^2) \tag{8.4}$$

$$\boxed{\psi = \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big)} \tag{8.5}$$

Это стандартная ZYX-декомпозиция (aerospace convention): ψ — курс, θ — тангаж, ϕ — крен.

8.3 Полярные координаты лазерного сканера → декартовы

dashboard_node.py, строки 516–521 — _scan_cb()

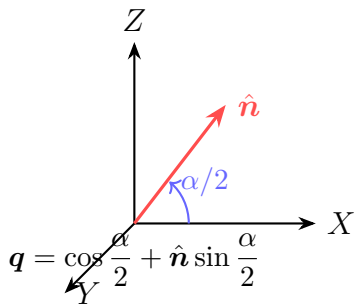
```
516 def _scan_cb(self, msg: LaserScan):
517     points, angle = [], msg.angle_min
518     for r in msg.ranges:
519         if msg.range_min < r < msg.range_max:
520             points.append([
521                 round(r * math.cos(angle), 3),
522                 round(r * math.sin(angle), 3)
523             ])
524         angle += msg.angle_increment
```

Полярные → декартовы координаты (точка облака лазерного сканера):

$$\begin{pmatrix} x \\ y \end{pmatrix} = r \begin{pmatrix} \cos\alpha \\ \sin\alpha \end{pmatrix} \tag{8.6}$$

где r — измеренная дальность, α — угол луча лазера.

8.4 Геометрическое представление кватерниона



8.5 Полная матрица вращения из кватерниона

$$\mathbf{R} = \begin{pmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{pmatrix} \tag{8.7}$$

Для чистого вращения вокруг Z ($q_x = q_y = 0$, $q_z = \sin(\psi/2)$, $q_w = \cos(\psi/2)$):

$$\mathbf{R}_z(\psi) = \begin{pmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{8.8}$$

Chapter 9

Калибровка IMU и цифровая фильтрация

Файл исходного кода

pi_nodes/nodes/imu_node.py (294 строки, Python)

9.1 Аппаратная часть: MPU6050

Датчик MPU6050 — 6-осевая IMU (3-осевой акселерометр + 3-осевой гироскоп) на шине I2C по адресу 0x68.

9.1.1 Масштабные коэффициенты

imu_node.py, строки 46–55

```
46 MPU6050_ADDR = 0x68
47 ACCEL_SCALE = 16384.0     # LSB/g at +/-2g
48 GYRO_SCALE  = 131.0       # LSB/(deg/s) at +/-250 deg/s
49 DEG2RAD = math.pi / 180.0
```

Перевод сырых показаний в физические единицы:

Акселерометр (из сырых 16-бит целых в м/с^2):

$$a_{\text{м/с}^2} = \frac{\text{raw}_{\text{16bit}}}{16384} \times 9.81 \tag{9.1}$$

Гироскоп (из сырых 16-бит целых в рад/с):

$$\omega_{\text{рад/с}} = \frac{\text{raw}_{\text{16bit}}}{131} \times \frac{\pi}{180} \tag{9.2}$$

imu_node.py, строки 133–149 — _read_raw()

```
raw = self._bus.read_i2c_block_data(MPU6050_ADDR, ACCEL_XOUT_H,
    14)
ax = struct.unpack('>h', bytes(raw[0:2]))[0] / ACCEL_SCALE * 9.81
ay = struct.unpack('>h', bytes(raw[2:4]))[0] / ACCEL_SCALE * 9.81
az = struct.unpack('>h', bytes(raw[4:6]))[0] / ACCEL_SCALE * 9.81
gx = struct.unpack('>h', bytes(raw[8:10]))[0] / GYRO_SCALE *
    DEG2RAD
gy = struct.unpack('>h', bytes(raw[10:12]))[0] / GYRO_SCALE *
    DEG2RAD
gz = struct.unpack('>h', bytes(raw[12:14]))[0] / GYRO_SCALE *
    DEG2RAD
```

Считываются 14 байт начиная с регистра 0x3B: три оси акселерометра (6 байт), температура (2 байта, пропускается), три оси гироскопа (6 байт). Формат — big-endian signed 16-bit.

9.2 Цифровой фильтр нижних частот (DLPF)

imu_node.py, строки 87–89

```
# DLPF mode 3: accel 44Hz, gyro 42Hz
bus.write_byte_data(MPU6050_ADDR, DLPF_CFG, 0x03)
bus.write_byte_data(MPU6050_ADDR, SMPLRT_DIV, 9)  # 1kHz/(1+9)=100
    Hz
```

Аппаратный DLPF на уровне сенсора: режим 3 обеспечивает полосу пропускания 44 Гц для акселерометра и 42 Гц для гироскопа, подавляя механические вибрации шасси.

Частота выборки:

$$f_{\text{sample}} = \frac{f_{\text{osc}}}{1 + \text{SMPLRT_DIV}} = \frac{1000}{1+9} = 100\,\text{Гц} \tag{9.3}$$

9.3 Экспоненциальное скользящее среднее (EMA)

imu_node.py, строки 60–64, 151–167 — _apply_ema()

```
60  ACCEL_EMA_ALPHA = 0.2      # alpha for accel (8 Hz cutoff)
61  GYRO_EMA_ALPHA  = 0.5      # alpha for gyro  (less smoothing)
```

```
160 self._ema_ax = a * ax + (1 - a) * self._ema_ax
161 self._ema_ay = a * ay + (1 - a) * self._ema_ay
162 self._ema_az = a * az + (1 - a) * self._ema_az
163 self._ema_gx = g * gx + (1 - g) * self._ema_gx
164 self._ema_gy = g * gy + (1 - g) * self._ema_gy
165 self._ema_gz = g * gz + (1 - g) * self._ema_gz
```

Рекуррентная формула EMA (фильтр первого порядка):

$$\boxed{\hat{x}_k = \alpha \cdot x_k + (1 - \alpha) \cdot \hat{x}_{k-1}} \tag{9.4}$$

Частота среза приблизительно:

$$f_{\text{cutoff}} \approx \frac{\alpha \cdot f_s}{2\pi(1 - \alpha)} \tag{9.5}$$

Для акселерометра ($\alpha = 0.2$, $f_s = 50$ Гц):

$$f_{\text{cutoff}} \approx \frac{0.2 \times 50}{2\pi \times 0.8} \approx 2.0\,\text{Гц} \tag{9.6}$$

Канал	α	**Частота среза**	**Назначение**
Акселерометр	0.2	≈2 Гц	Сильное подавление вибраций
Гироскоп	0.5	≈8 Гц	Умеренное сглаживание

9.4 Калибровка при старте

imu_node.py, строки 169–212 — _finish_calibration()

```
n = len(self._cal_samples)   # 100 samples (2 sec @ 50 Hz)
sum_ax = sum_ay = sum_az = 0.0
sum_gx = sum_gy = sum_gz = 0.0
for (ax, ay, az, gx, gy, gz) in self._cal_samples:
    sum_ax += ax; sum_ay += ay; sum_az += az
    sum_gx += gx; sum_gy += gy; sum_gz += gz

# Gyro offset: average at rest should be zero
self._gyro_offset = (sum_gx / n, sum_gy / n, sum_gz / n)

# Gravity vector in body frame at rest
avg_ax, avg_ay, avg_az = sum_ax/n, sum_ay/n, sum_az/n
self._gravity_body = (avg_ax, avg_ay, avg_az)

# Initial orientation from gravity
self._home_roll  = math.atan2(avg_ay, avg_az)
self._home_pitch = math.atan2(-avg_ax,
                    math.sqrt(avg_ay*avg_ay + avg_az*avg_az))
```

9.4.1 Смещение нуля гироскопа

В покое идеальный гироскоп показывает $\omega = 0$. В реальности имеется постоянное смещение (bias):

$$\boldsymbol{b}_{\text{gyro}} = \frac{1}{N}\sum_{k=1}^{N} \boldsymbol{\omega}_k^{\text{raw}}, \quad N = 100 \tag{9.7}$$

Все последующие измерения корректируются: $\boldsymbol{\omega}_{\text{corr}} = \boldsymbol{\omega}_{\text{raw}} - \boldsymbol{b}_{\text{gyro}}$.

9.4.2 Вектор гравитации в системе координат тела

Среднее показание акселерометра в покое — это вектор гравитации в системе координат IMU:

$$\boldsymbol{g}_{\text{body}} = \frac{1}{N}\sum_{k=1}^{N} \boldsymbol{a}_k^{\text{raw}} = \left(\bar{a}_x,\, \bar{a}_y,\, \bar{a}_z\right) \tag{9.8}$$

9.4.3 Начальная ориентация (home)

Из вектора гравитации вычисляется начальный наклон:

Крен:

$$\phi_{\text{home}} = \operatorname{atan2}(\bar{a}_y,\, \bar{a}_z) \tag{9.9}$$

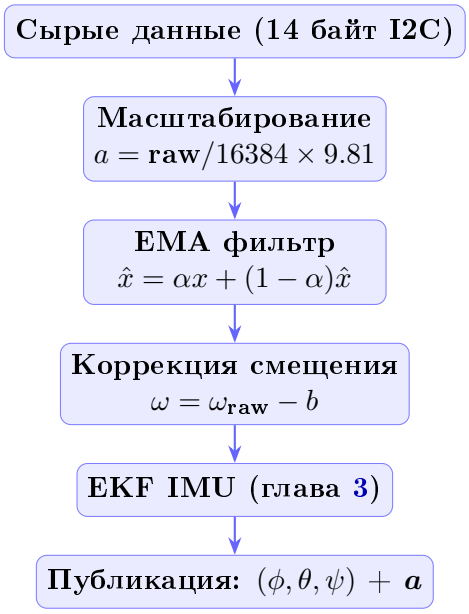
Тангаж:

$$\theta_{\text{home}} = \operatorname{atan2}\left(-\bar{a}_x,\, \sqrt{\bar{a}_y^2 + \bar{a}_z^2}\right) \tag{9.10}$$

Модуль вектора гравитации (для диагностики):

$$|\boldsymbol{g}| = \sqrt{\bar{a}_x^2 + \bar{a}_y^2 + \bar{a}_z^2} \approx 9.81\,\text{м/с}^2 \tag{9.11}$$

9.5 Конвейер обработки IMU



9.6 Сводная таблица параметров

Параметр	Значение	Описание
ACCEL_SCALE	16384	LSB/g при $\pm 2\,g$
GYRO_SCALE	131	LSB/(°/с) при $\pm 250\,°/\text{с}$
DLPF_CFG	3	Полоса: 44 Гц акс., 42 Гц гиро
SMPLRT_DIV	9	Частота: 100 Гц
ACCEL_EMA_ALPHA	0.2	Сглаживание акселерометра
GYRO_EMA_ALPHA	0.5	Сглаживание гироскопа
CALIBRATION_SAMPLES	100	2 сек калибровки

9.7 C++ реализация (оптимизированная)

Файл исходного кода

ros_ws/src/robot_pkg_cpp/src/imu_node.cpp (172 строки, C++)

imu_node.cpp, строки 90–127 — readRaw()

```
// Bytes 0-5: accelerometer (X,Y,Z)
double ax = to_int16(raw[0],  raw[1])  / ACCEL_SCALE * GRAVITY;
double ay = to_int16(raw[2],  raw[3])  / ACCEL_SCALE * GRAVITY;
double az = to_int16(raw[4],  raw[5])  / ACCEL_SCALE * GRAVITY;
// Bytes 8-13: gyroscope (X,Y,Z)
double gx = to_int16(raw[8],  raw[9])  / GYRO_SCALE * DEG2RAD;
double gy = to_int16(raw[10], raw[11]) / GYRO_SCALE * DEG2RAD;
double gz = to_int16(raw[12], raw[13]) / GYRO_SCALE * DEG2RAD;
```

C++ версия использует прямой Linux I2C ioctl вместо Python smbus2. Те же масштабные коэффициенты, тот же формат данных. Преимущество: обратный вызов таймера $\approx 10\,\mu$с vs ≈ 1 мс в Python (отсутствие GIL и интерпретатора).

9.8 Теоретическое обоснование: цифровая фильтрация

9.8.1 EMA как дискретный ФНЧ

Экспоненциальное скользящее среднее (EMA) — это **дискретный фильтр низких частот первого порядка** (IIR-фильтр):

$$y_k = \alpha\, x_k + (1-\alpha)\, y_{k-1} \tag{9.12}$$

Z-преобразование:

$$H(z) = \frac{\alpha}{1-(1-\alpha)z^{-1}} \tag{9.13}$$

Полюс фильтра: $z_p = 1-\alpha$. Условие устойчивости $|z_p| < 1$ выполняется при $0 < \alpha < 2$.

9.8.2 Частотная характеристика EMA

Частота среза (на уровне -3 дБ):

$$f_c = \frac{f_s}{2\pi} \arccos\left(\frac{2-\alpha}{2(1-\alpha)}\right) \tag{9.14}$$

Для $\alpha = 0.15$, $f_s = 50$ Гц: $f_c \approx 1.2$ Гц. Это подавляет вибрации двигателей (> 10 Гц), сохраняя полезный сигнал поворота (< 1 Гц).

9.8.3 Связь с аналоговым RC-фильтром

EMA эквивалентен дискретизации аналогового RC-фильтра:

$$W(p) = \frac{1}{\tau p + 1}, \quad \tau = \frac{(1-\alpha)\Delta t}{\alpha} \tag{9.15}$$

Для $\alpha = 0.15$: $\tau = 0.85 \cdot 0.02/0.15 \approx 0.113\,\text{с}$.

9.8.4 DLPF как аппаратный ФНЧ

Цифровой фильтр низких частот (DLPF) MPU-6050 реализует аналогичную функцию аппаратно, с полосой 44 Гц (режим 3). Два последовательных ФНЧ (DLPF + EMA) дают затухание -12 дБ/окт (второй порядок) в полосе подавления.

Chapter 10

A* Pathfinding и алгоритм Брезенхэма

Файл исходного кода	
compute_node/simulator.py	(строки 103–280, Python)

10.1 Постановка задачи

Робот должен найти оптимальный путь от текущей позиции до цели, обходя стены арены и запретные зоны. Используется **A*** — классический информированный алгоритм поиска на графе.

10.2 Построение сетки занятости

simulator.py, строки 109–137 — _build_grid()

```
109 def _build_grid(arena, zones, robot_radius):
110     """Build occupancy grid: True = blocked, False = free."""
111     cols = int(arena.width / PATH_GRID_RES)   # 0.05 m/cell
112     rows = int(arena.height / PATH_GRID_RES)
113     grid = [[False] * cols for _ in range(rows)]
114     margin = robot_radius + PATH_SAFETY_MARGIN  # 0.12 + 0.10 =
            0.22 m
115
116     for r in range(rows):
117         for c in range(cols):
118             wx = (c + 0.5) * PATH_GRID_RES
119             wy = (r + 0.5) * PATH_GRID_RES
120             # Block near arena walls
121             if (wx < margin or wx > arena.width - margin or
122                     wy < margin or wy > arena.height - margin):
123                 grid[r][c] = True
124                 continue
125             # Block inside forbidden zones (with safety margin)
126             for z in zones:
127                 zx1, zy1 = z['x1'] - margin, z['y1'] - margin
128                 zx2, zy2 = z['x2'] + margin, z['y2'] + margin
129                 if zx1 <= wx <= zx2 and zy1 <= wy <= zy2:
130                     grid[r][c] = True
131                     break
132     return grid, rows, cols
```

Арена дискретизируется на ячейки $\delta = 5$ см:

$$\text{cols} = \left\lfloor \frac{W}{\delta} \right\rfloor, \quad \text{rows} = \left\lfloor \frac{H}{\delta} \right\rfloor \tag{10.1}$$

Зона безопасности вокруг каждого препятствия — **раздутие Минковского**:

$$\text{margin} = r_{\text{robot}} + r_{\text{safety}} = 0.12 + 0.10 = 0.22\,\text{м} \tag{10.2}$$

10.3 Преобразование координат

simulator.py, строки 140–147

```
140 def _world_to_grid(wx, wy):
141     return int(wx / PATH_GRID_RES), int(wy / PATH_GRID_RES)
142
143 def _grid_to_world(c, r):
144     return (c + 0.5) * PATH_GRID_RES, (r + 0.5) * PATH_GRID_RES
```

Мир → сетка:

$$c = \left\lfloor \frac{w_x}{\delta} \right\rfloor, \quad r = \left\lfloor \frac{w_y}{\delta} \right\rfloor \tag{10.3}$$

Сетка → мир (центр ячейки):

$$w_x = (c + 0.5)\,\delta, \quad w_y = (r + 0.5)\,\delta \tag{10.4}$$

10.4 Алгоритм A*

simulator.py, строки 150–213 — find_path()

```
173 # A* with 8-directional movement
174 DIRS  = [(-1,0),(1,0),(0,-1),(0,1), (-1,-1),(-1,1),(1,-1),(1,1)]
175 COSTS = [1.0, 1.0, 1.0, 1.0,  1.414, 1.414, 1.414, 1.414]
176
177 def heuristic(r1, c1, r2, c2):
178     dr = abs(r1 - r2)
179     dc = abs(c1 - c2)
180     return max(dr, dc) + 0.414 * min(dr, dc)  # octile distance
181
182 open_set = [(heuristic(sr, sc, gr, gc), 0.0, sr, sc)]
183 g_cost = {(sr, sc): 0.0}
184 came_from = {}
185
186 while open_set:
187     _f, g, r, c = heapq.heappop(open_set)
188     if r == gr and c == gc:
189         # Reconstruct path ...
190         break
191     for (dr, dc), cost in zip(DIRS, COSTS):
192         nr, nc = r + dr, c + dc
193         if 0 <= nr < rows and 0 <= nc < cols and not grid[nr][nc]:
194             ng = g + cost
195             if ng < g_cost.get((nr, nc), float('inf')):
196                 g_cost[(nr, nc)] = ng
197                 f = ng + heuristic(nr, nc, gr, gc)
198                 came_from[(nr, nc)] = (r, c)
199                 heapq.heappush(open_set, (f, ng, nr, nc))
```

10.4.1 Оценочная функция A*

A* расширяет вершину с минимальным f:

$$\boxed{f(n) = g(n) + h(n)} \tag{10.5}$$

где $g(n)$ — стоимость пути от старта до n, $h(n)$ — эвристика от n до цели.

10.4.2 Октильная эвристика

Для 8-связного графа (с диагоналями) оптимальная допустимая эвристика:

$$\boxed{h(n) = \max(\Delta r, \Delta c) + (\sqrt{2} - 1)\,\min(\Delta r, \Delta c)} \tag{10.6}$$

где $\Delta r = |r_n - r_g|$, $\Delta c = |c_n - c_g|$. В коде $\sqrt{2} - 1 \approx 0.414$.

Свойства:

- **Допустимая** (admissible): $h(n) \le h^*(n)$ — не переоценивает.
- **Монотонная** (consistent): $h(n) \le c(n, n') + h(n')$ — гарантирует оптимальность без повторного посещения.

10.4.3 Стоимость переходов

$$c(n, n') = \begin{cases} 1.0 & \text{кардинальное направление (4 стороны)} \\ \sqrt{2} \approx 1.414 & \text{диагональное направление (4 угла)} \end{cases} \tag{10.7}$$

10.5 Алгоритм Брезенхэма (Line-of-Sight)

simulator.py, строки 234–258 — _line_of_sight()

```
234 def _line_of_sight(x0, y0, x1, y1, grid, rows, cols):
235     """Bresenham: True if straight line is obstacle-free."""
236     c0, r0 = _world_to_grid(x0, y0)
237     c1, r1 = _world_to_grid(x1, y1)
238     dc = abs(c1 - c0)
239     dr = abs(r1 - r0)
240     sc = 1 if c0 < c1 else -1
241     sr = 1 if r0 < r1 else -1
242     err = dc - dr
243     while True:
244         if grid[r0][c0]:
245             return False          # blocked cell
246         if r0 == r1 and c0 == c1:
247             break
248         e2 = 2 * err
249         if e2 > -dr:
250             err -= dr
251             c0 += sc
252         if e2 < dc:
253             err += dc
254             r0 += sr
255     return True
```

Алгоритм Брезенхэма растеризует отрезок прямой на целочисленной сетке:

$$\text{err} = \Delta c - \Delta r, \quad e_2 = 2 \cdot \text{err} \tag{10.8}$$

На каждом шаге:

- Если $e_2 > -\Delta r$: $\text{err} \leftarrow \text{err} - \Delta r$, сдвиг по c.

- Если $e_2 < \Delta c$: $\text{err} \leftarrow \text{err} + \Delta c$, сдвиг по r.

Алгоритм работает за $O(\max(\Delta c, \Delta r))$ целочисленных операций (без деления и плавающей точки).

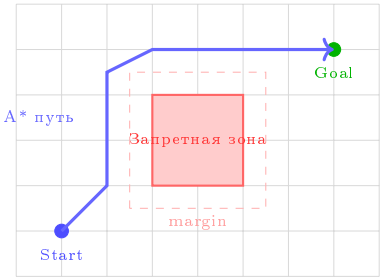
10.6 Сглаживание пути (Line-of-Sight Pruning)

simulator.py, строки 261–279 — _smooth_path()

```
261 def _smooth_path(path, grid, rows, cols):
262     """Reduce points via line-of-sight pruning."""
263     smoothed = [path[0]]
264     i = 0
265     while i < len(path) - 1:
266         best = i + 1
267         for j in range(len(path) - 1, i + 1, -1):
268             if _line_of_sight(path[i][0], path[i][1],
269                               path[j][0], path[j][1],
270                               grid, rows, cols):
271                 best = j
272                 break
273         smoothed.append(path[best])
274         i = best
275     return smoothed
```

Удаляет промежуточные точки, если прямая видимость (Bresenham check) свободна. Уменьшает число путевых точек с $O(N)$ до $O(\sqrt{N})$.

10.7 Геометрия A* на сетке



10.8 Сводная таблица параметров

Параметр	Значение	Описание
PATH_GRID_RES	0.05 м	Размер ячейки сетки
PATH_SAFETY_MARGIN	0.10 м	Запас безопасности
ROBOT_RADIUS	0.12 м	Радиус робота
Кардинальная стоимость	1.0	4 направления
Диагональная стоимость	$\sqrt{2} \approx 1.414$	4 диагонали

Chapter 11

Физический симулятор: кинематика, сенсоры, камера

Файл исходного кода

compute_node/simulator.py (строки 282–588, Python)

11.1 Кинематика робота (SimRobot)

simulator.py, строки 373–397 — tick()

```
373 def tick(self, dt: float):
374     self._prev_v_linear = self.v_linear
375     prev_x, prev_y = self.x, self.y
376     # Integrate velocities (unicycle model)
377     self.x += self.v_linear * math.cos(self.theta) * dt
378     self.y += self.v_linear * math.sin(self.theta) * dt
379     self.theta += self.v_angular * dt
380     # Normalise theta
381     self.theta = math.atan2(math.sin(self.theta), math.cos(self.
        theta))
382     # Wall collision -- clamp inside arena
383     margin = ROBOT_RADIUS
384     self.x = max(margin, min(self.arena.width - margin, self.x))
385     self.y = max(margin, min(self.arena.height - margin, self.y))
386     # Forbidden zone collision -- push back
387     for z in self.arena.forbidden_zones:
388         zx1 = z['x1'] - margin
389         zy1 = z['y1'] - margin
390         zx2 = z['x2'] + margin
391         zy2 = z['y2'] + margin
392         if zx1 <= self.x <= zx2 and zy1 <= self.y <= zy2:
393             self.x, self.y = prev_x, prev_y
394             self.v_linear = 0.0
395             break
```

11.1.1 Дифференциальные уравнения кинематики

Робот моделируется как **unicycle** — одноколейная модель с линейной и угловой скоростями:

$$\dot{x} = v\cos\theta \tag{11.1}$$
$$\dot{y} = v\sin\theta \tag{11.2}$$
$$\dot{\theta} = \omega \tag{11.3}$$

Дискретизация методом Эйлера ($\Delta t = 0.05\,\text{с}$):

$$\boxed{\begin{pmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \\ \theta_k \end{pmatrix} + \Delta t \begin{pmatrix} v\cos\theta_k \\ v\sin\theta_k \\ \omega \end{pmatrix}} \tag{11.4}$$

11.1.2 Нормализация угла

Для предотвращения накопления $\theta > 2\pi$:

$$\theta \leftarrow \operatorname{atan2}\bigl(\sin\theta,\, \cos\theta\bigr) \tag{11.5}$$

11.1.3 Коллизия со стенами

Ограничение позиции внутри арены:

$$x \leftarrow \operatorname{clamp}(x,\ r_{\text{robot}},\ W - r_{\text{robot}}), \quad y \leftarrow \operatorname{clamp}(y,\ r_{\text{robot}},\ H - r_{\text{robot}}) \tag{11.6}$$

11.2 Ультразвуковой сенсор (SimSensors)

simulator.py, строки 417–464 — _update_ultrasonic()

```
417 def _update_ultrasonic(self, robot, arena):
418     """Ray-cast forward from robot."""
419     rx, ry = robot.x, robot.y
420     dx = math.cos(robot.theta)
421     dy = math.sin(robot.theta)
422     best = ULTRASONIC_MAX
423
424     # Check walls (ray-plane intersection)
425     if dx > 0:
426         t = (arena.width - rx) / dx      # right wall
427         if ULTRASONIC_MIN < t < best: best = t
428     if dx < 0:
429         t = -rx / dx                     # left wall
430         if ULTRASONIC_MIN < t < best: best = t
431     if dy > 0:
432         t = (arena.height - ry) / dy     # top wall
433         if ULTRASONIC_MIN < t < best: best = t
434     if dy < 0:
435         t = -ry / dy                     # bottom wall
436         if ULTRASONIC_MIN < t < best: best = t
437
438     # Check balls (ray-sphere intersection)
439     for ball in arena.balls:
440         if ball['grabbed']: continue
441         to_ball_x = ball['x'] - rx
442         to_ball_y = ball['y'] - ry
443         proj = to_ball_x * dx + to_ball_y * dy   # projection
444         perp = abs(to_ball_x * dy - to_ball_y * dx)  #
                perpendicular
445         if proj > ULTRASONIC_MIN and perp < ball['radius'] + 0.05:
446             if proj < best: best = proj
447
448     # Gaussian noise
449     noise = random.gauss(0, 0.005)
450     self.range_m = max(ULTRASONIC_MIN, min(ULTRASONIC_MAX, best +
            noise))
```

11.2.1 Пересечение луч–плоскость (стены)

Луч из точки (r_x, r_y) по направлению $(\cos\theta, \sin\theta)$:

$$\boldsymbol{p}(t) = \begin{pmatrix} r_x \\ r_y \end{pmatrix} + t \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}, \quad t > 0 \tag{11.7}$$

Расстояние до стены $x = W$ (при $\cos\theta > 0$):

$$t = \frac{W - r_x}{\cos\theta} \tag{11.8}$$

11.2.2 Пересечение луч–сфера (шарики)

Для каждого шарика с центром (b_x, b_y):

Проекция вектора “к шарику” на направление луча:

$$t_{\text{proj}} = (\boldsymbol{b} - \boldsymbol{r}) \cdot \hat{\boldsymbol{d}} = (b_x - r_x)\cos\theta + (b_y - r_y)\sin\theta \tag{11.9}$$

Перпендикулярное расстояние от луча до центра шарика:

$$d_\perp = |(b_x - r_x)\sin\theta - (b_y - r_y)\cos\theta| \tag{11.10}$$

Условие обнаружения: $d_\perp < r_{\text{ball}} + 0.05$ (конус ультразвука).

11.2.3 Шумовая модель

$$r_{\text{meas}} = \text{clamp}(r_{\text{true}} + \mathcal{N}(0, 0.005^2),\ r_{\min},\ r_{\max}) \tag{11.11}$$

11.3 Симуляция IMU

simulator.py, строки 466–472 — _update_imu()

```
466 def _update_imu(self, robot):
467     noise = random.gauss(0, 0.3)
468     self.imu_yaw = math.degrees(robot.theta) + noise
469     self.imu_pitch = random.gauss(0, 0.2)
470     self.imu_roll  = random.gauss(0, 0.2)
471     self.imu_gyro_z = robot.v_angular
472     self.accel_x = (robot.v_linear - robot._prev_v_linear) /
            SIM_DT
```

Линейное ускорение из конечной разности скорости:

$$a_x = \frac{v_k - v_{k-1}}{\Delta t} \tag{11.12}$$

11.4 Модель перспективной камеры (SimDetector)

simulator.py, строки 510–543 — _render_camera()

```
514 # Vector from robot to ball
515 dx = bx - robot.x
516 dy = by - robot.y
517 dist = math.sqrt(dx*dx + dy*dy)
518
519 # Angle to ball relative to robot heading
520 angle = math.atan2(dy, dx) - robot.theta
521 angle = math.atan2(math.sin(angle), math.cos(angle))
522
523 if abs(angle) > CAM_FOV / 2:
524     continue   # outside field of view
525
526 # Project to screen
527 screen_x = int(CAM_W/2 + (angle / (CAM_FOV/2)) * (CAM_W/2))
528
529 # Apparent size (pinhole camera model)
530 apparent_px = int(FOCAL_LENGTH_PX * BALL_DIAMETER_M / dist)
531
532 # Vertical position (depth cue)
533 screen_y = horizon_y + int((1.0 - 0.03/max(0.1, dist)) *
534                            (CAM_H - horizon_y) * 0.7)
535
536 # Confidence proportional to distance
537 conf = max(0.5, min(0.99, 1.0 - dist / 3.0))
```

11.4.1 Проверка видимости (Field of View)

Угол на объект относительно курса робота:

$$\alpha = \text{atan2}(b_y - r_y,\, b_x - r_x) - \theta_{\text{robot}} \tag{11.13}$$

Объект виден если: $|\alpha| \leq \text{FOV}/2 = 30°$.

11.4.2 Горизонтальная проекция на экран

$$\boxed{s_x = \frac{W}{2} + \frac{\alpha}{\text{FOV}/2} \cdot \frac{W}{2}} \tag{11.14}$$

Это линейная проекция: угол $\alpha = 0$ — центр кадра, $\pm\text{FOV}/2$ — края.

11.4.3 Модель pinhole-камеры (размер объекта)

Видимый размер объекта на сенсоре:

$$\boxed{s_{\text{px}} = \frac{f \cdot D_{\text{real}}}{d}} \tag{11.15}$$

где $f = 500\,\text{пкс}$ — фокусное расстояние, $D_{\text{real}} = 0.04\,\text{м}$ — диаметр шарика, d — расстояние до объекта.

Примечание

Это обратная зависимость: при удвоении расстояния видимый размер уменьшается вдвое. Тот же принцип используется в follow_me_node.py (глава 12) для оценки дистанции до человека.

11.4.4 Вертикальная позиция (глубинная подсказка)

Объекты на полу: чем ближе к роботу, тем ниже на экране:

$$s_y = h_{\text{горизонт}} + \left(1 - \frac{0.03}{\max(0.1, d)}\right) \cdot 0.7(H - h_{\text{горизонт}}) \tag{11.16}$$

11.4.5 Модель доверия (confidence)

$$\text{conf} = \text{clamp}\left(1 - \frac{d}{3.0},\ 0.5,\ 0.99\right) \tag{11.17}$$

11.5 Параметры симулятора

Параметр	Значение	Описание
ARENA_W, H	$3.0 \times 3.0\,\text{м}$	Размер арены
SIM_DT	$0.05\,\text{с}$	Шаг интегрирования (20 Гц)
MAX_LINEAR	$0.3\,\text{м/с}$	Макс. линейная скорость
MAX_ANGULAR	$2.0\,\text{рад/с}$	Макс. угловая скорость
ROBOT_RADIUS	$0.12\,\text{м}$	Радиус робота
BALL_DIAMETER_M	$0.04\,\text{м}$	Диаметр шарика
FOCAL_LENGTH_PX	$500\,\text{пкс}$	Фокусное расстояние камеры
CAM_FOV	$60°$	Поле зрения камеры
CAM_W, H	$640 \times 480\,\text{пкс}$	Разрешение камеры
ULTRASONIC_MAX	$2.0\,\text{м}$	Макс. дальность УЗ

Chapter 12

Следование за человеком: оценка дистанции и P-регулятор

Файл исходного кода

ros_ws/src/robot_pkg/robot_pkg/follow_me_node.py (138 строк, Python)

12.1 Постановка задачи

Модуль обнаруживает человека по детекции YOLO (класс “person”) и управляет роботом для поддержания заданной дистанции следования.

12.2 Оценка расстояния по bounding box

follow_me_node.py, строки 83–88

```
83 # Estimate distance from bounding box height
84 bbox_h = best.get('h', 1)
85 if bbox_h > 10:
86     self._person_distance = (PERSON_HEIGHT_M * FOCAL_LENGTH_PX) /
       bbox_h
87 else:
88     self._person_distance = TARGET_DISTANCE
```

Используется **модель pinhole-камеры** (та же, что в главе 11):

$$h_{\text{px}} = \frac{f \cdot H_{\text{real}}}{d} \quad \Rightarrow \quad \boxed{d = \frac{H_{\text{person}} \cdot f}{h_{\text{bbox}}}} \tag{12.1}$$

где:

- $H_{\text{person}} = 1.7\,\text{м}$ — средний рост человека
- $f = 500\,\text{пкс}$ — фокусное расстояние камеры

- h_{bbox} — высота bounding box в пикселях

12.3 Рулевое управление (P-регулятор по углу)

follow_me_node.py, строки 90–103

```
90  # Steering: centre person in frame
91  person_cx = best.get('x', IMG_CX) + best.get('w', 0) / 2.0
92  error_x = (person_cx - IMG_CX) / IMG_CX    # normalized -1 to +1
93
94  # Speed: proportional to distance error
95  dist_error = self._person_distance - TARGET_DISTANCE
96
97  twist = Twist()
98  twist.angular.z = -error_x * 0.8           # angular P-controller
99  if dist_error > 0.15:
100     twist.linear.x = min(0.20, dist_error * 0.3)
101 elif dist_error < -0.15:
102     twist.linear.x = max(-0.10, dist_error * 0.2)
```

12.3.1 Нормализованная ошибка центрирования

$$e_x = \frac{c_{\text{person}} - c_{\text{image}}}{c_{\text{image}}} \in [-1, +1] \tag{12.2}$$

где $c_{\text{image}} = 320\,\text{пкс}$ — центр изображения.

12.3.2 Угловая скорость (P-регулятор)

$$\omega = -K_\omega \cdot e_x, \quad K_\omega = 0.8 \tag{12.3}$$

Знак «−» обеспечивает: человек справа ($e_x > 0$) ⇒ робот поворачивает вправо ($\omega < 0$).

12.3.3 Линейная скорость (P-регулятор с мёртвой зоной)

Ошибка дистанции:

$$e_d = d_{\text{measured}} - d_{\text{target}}, \quad d_{\text{target}} = 1.0\,\text{м} \tag{12.4}$$

$$v = \begin{cases} \min(0.20,\ 0.3 \cdot e_d) & \text{если } e_d > 0.15\,\text{м (далеко)} \\ \max(-0.10,\ 0.2 \cdot e_d) & \text{если } e_d < -0.15\,\text{м (близко)} \\ 0 & \text{иначе (мёртвая зона)} \end{cases} \tag{12.5}$$

Мёртвая зона $\pm 0.15\,\text{м}$ предотвращает колебания вокруг целевой дистанции.

12.4 Теоретическое обоснование: замкнутая система слежения

12.4.1 Двухконтурная структура

Система Follow-Me реализует **двухконтурный** регулятор (гл. 1, раздел 1.7):

1. **Угловой контур**: $\omega = -K_\omega e_x$ — центрирование человека в кадре
2. **Дистанционный контур**: $v = K_v e_d$ — поддержание заданной дистанции

12.4.2 Передаточные функции контуров

Для углового контура (робот — интегратор $\dot{\theta} = \omega$):

$$W_\omega(p) = \frac{K_\omega}{p + K_\omega} = \frac{0.8}{p + 0.8}, \quad \tau_\omega = \frac{1}{K_\omega} = 1.25\,\text{с} \tag{12.6}$$

Для дистанционного контура (робот — интегратор $\dot{d} = -v$):

$$W_v(p) = \frac{K_v}{p + K_v} = \frac{0.3}{p + 0.3}, \quad \tau_v = \frac{1}{K_v} = 3.3\,\text{с} \tag{12.7}$$

Угловой контур быстрее дистанционного ($\tau_\omega < \tau_v$) — сначала робот поворачивается к человеку, затем подъезжает.

12.4.3 Мёртвая зона и устойчивость

Мёртвая зона ± 0.15 м предотвращает **автоколебания** (предельный цикл), которые возникали бы из-за дискретности и задержки обработки YOLO-детекции. Это аналог **гистерезиса** в релейных системах управления.

12.5 Таймаут потери цели

follow_me_node.py, строки 106–112

```
106 def _tick(self):
107     if self._tracking and (time.time() - self._last_person_time) >
            LOST_TIMEOUT:
108         self._tracking = False
109         self._cmd_pub.publish(Twist())  # stop
```

Если человек не обнаружен в течение $t_{\text{timeout}} = 2.0$ с, робот останавливается. Это защита от неконтролируемого движения.

12.6 Сводная таблица параметров

Параметр	**Значение**	**Описание**
TARGET_DISTANCE	1.0 м	Целевая дистанция следования
PERSON_HEIGHT_M	1.7 м	Средний рост человека
FOCAL_LENGTH_PX	500 пкс	Фокусное расстояние камеры
IMG_CX	320 пкс	Центр изображения по x
K_ω	0.8	Коэффициент P-регулятора угла
K_v^+	0.3	Коэффициент скорости (далеко)
K_v^-	0.2	Коэффициент скорости (близко)
LOST_TIMEOUT	2.0 с	Таймаут потери цели

Chapter 13

Патрулирование: навигация по точкам и кватернионы

Файл исходного кода

ros_ws/src/robot_pkg/robot_pkg/patrol_node.py (145 строк, Python)

13.1 Постановка задачи

Модуль реализует циклический обход набора путевых точек (waypoints) с отправкой целей в Nav2 (ROS2 навигационный стек).

13.2 Проверка достижения точки

patrol_node.py, строки 90–108 — _tick()

```
# Check if reached
dist = math.hypot(tx - self._robot_x, ty - self._robot_y)
if dist < GOAL_TOLERANCE:    # 0.20 m
    self._current_idx = (self._current_idx + 1) % len(self.
        _waypoints)
    self._goal_sent = False
```

Евклидово расстояние до цели:

$$d = \sqrt{(t_x - x_r)^2 + (t_y - y_r)^2} = \mathrm{hypot}(\Delta x, \Delta y) \tag{13.1}$$

Точка считается достигнутой при $d < 0.20\,\text{м}$.

Циклическая индексация:

$$i_{\text{next}} = (i_{\text{curr}} + 1) \bmod N_{\text{waypoints}} \tag{13.2}$$

13.3 Формирование цели Nav2 (кватернион из yaw)

patrol_node.py, строки 110–118 — _send_goal()

```
110 def _send_goal(self, x, y, yaw):
111     goal = PoseStamped()
112     goal.header.stamp = self.get_clock().now().to_msg()
113     goal.header.frame_id = 'map'
114     goal.pose.position.x = x
115     goal.pose.position.y = y
116     goal.pose.orientation.z = math.sin(yaw / 2.0)
117     goal.pose.orientation.w = math.cos(yaw / 2.0)
118     self._goal_pub.publish(goal)
```

Кватернион из угла курса (чистое вращение вокруг Z):

$$\boxed{q_z = \sin\left(\frac{\psi}{2}\right), \quad q_w = \cos\left(\frac{\psi}{2}\right), \quad q_x = q_y = 0} \tag{13.3}$$

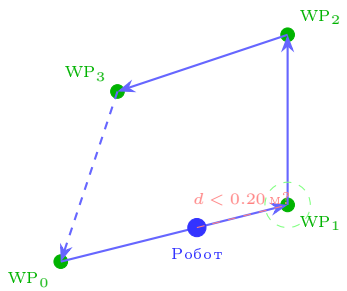
Это следствие формулы Родрига: для оси вращения $\hat{\boldsymbol{n}} = (0, 0, 1)$ (ось Z, вертикальная):

$$\boldsymbol{q} = \cos\frac{\psi}{2} + \hat{\boldsymbol{n}}\sin\frac{\psi}{2} = \left(0,\ 0,\ \sin\frac{\psi}{2},\ \cos\frac{\psi}{2}\right) \tag{13.4}$$

Примечание

ROS2 использует порядок (q_x, q_y, q_z, q_w) в сообщениях geometry_msgs/Quaternion. Для 2D-робота с вращением только вокруг Z достаточно задать q_z и q_w, остальные равны нулю.

13.4 Геометрия патрулирования



13.5 Параметры

Параметр	**Значение**	**Описание**
GOAL_TOLERANCE	0.20 м	Допуск достижения точки
Частота обновления	2 Гц	Частота проверки цели
Кватернион	$(0, 0, \sin\frac{\psi}{2}, \cos\frac{\psi}{2})$	Ориентация цели

Chapter 14

Геозона, преобразование сенсоров и реактивная навигация

14.1 Геозона: алгоритм Point-in-Polygon

Файл исходного кода

compute_node/geofence_map_publisher.py (108 строк, Python)

geofence_map_publisher.py, строки 79–91 — _point_in_polygon()

```
79  @staticmethod
80  def _point_in_polygon(x, y, poly):
81      """Ray-casting algorithm."""
82      n = len(poly)
83      inside = False
84      j = n - 1
85      for i in range(n):
86          xi, yi = poly[i]
87          xj, yj = poly[j]
88          if ((yi > y) != (yj > y)) and \
89             (x < (xj - xi) * (y - yi) / (yj - yi) + xi):
90              inside = not inside
91          j = i
92      return inside
```

14.1.1 Алгоритм трассировки луча (Ray Casting)

Для определения, находится ли точка (x, y) внутри полигона, из неё проводится горизонтальный луч вправо и считается число пересечений с рёбрами:

$$\text{inside} = \big(\text{число пересечений}\big) \bmod 2 = 1 \tag{14.1}$$

Для каждого ребра $(x_i, y_i) \to (x_j, y_j)$:

1. Проверка пересечения по y:

$$(y_i > y) \neq (y_j > y) \quad \text{(ребро пересекает горизонталь } y\text{)} \tag{14.2}$$

2. Вычисление x-координаты пересечения (линейная интерполяция):

$$x_{\text{cross}} = x_i + \frac{(x_j - x_i)(y - y_i)}{y_j - y_i} \tag{14.3}$$

3. Если $x < x_{\text{cross}}$ — луч пересекает ребро, инвертируем флаг.

14.1.2 Построение карты занятости

geofence_map_publisher.py, строки 46–77

```
47 poly = np.array(self._polygon, dtype=np.float64)
48 x_min = poly[:, 0].min() - self._padding   # 0.5 m padding
49 x_max = poly[:, 0].max() + self._padding
50 width  = int((x_max - x_min) / self._resolution)  # 0.05 m/cell
51 height = int((y_max - y_min) / self._resolution)
52
53 # 0=free inside, 100=lethal outside
54 grid = np.full((height, width), 100, dtype=np.int8)
55 for iy in range(height):
56     for ix in range(width):
57         px = x_min + ix * self._resolution
58         py = y_min + iy * self._resolution
59         if self._point_in_polygon(px, py, poly):
60             grid[iy, ix] = 0
```

Карта создаётся как OccupancyGrid (ROS2 nav_msgs):

- 0 — свободная ячейка (внутри полигона)
- 100 — летальное препятствие (за пределами арены)

14.2 Ультразвук → LaserScan: синтетическое сканирование

Файл исходного кода

compute_node/depth_to_scan_node.py (82 строки, Python)

depth_to_scan_node.py, строки 43–65

```
43 def _publish_scan(self):
44     scan = LaserScan()
45     scan.angle_min = -math.pi / 2.0          # -90 deg
46     scan.angle_max =  math.pi / 2.0          # +90 deg
47     scan.angle_increment = (ANGLE_MAX - ANGLE_MIN) / NUM_BEAMS  #
           180 beams
48     scan.range_min = 0.02
49     scan.range_max = 3.0
50
51     ranges = [MAX_RANGE] * NUM_BEAMS          # all unknown
52     centre = NUM_BEAMS // 2
53     # Spread ultrasonic over ~15 deg cone
54     cone_beams = int(0.26 / scan.angle_increment)
55     for i in range(max(0, centre - cone_beams),
56                    min(NUM_BEAMS, centre + cone_beams + 1)):
57         ranges[i] = min(self._latest_range, MAX_RANGE)
58     scan.ranges = ranges
```

14.2.1 Геометрия конуса ультразвука

Один ультразвуковой датчик HC-SR04 даёт единственное расстояние с конусом обзора ≈30°. Для совместимости с slam_toolbox это значение «размазывается» по нескольким лучам:

$$\Delta\alpha = \frac{\alpha_{\max} - \alpha_{\min}}{N_{\text{beams}}} = \frac{\pi}{180} \approx 0.0175\,\text{рад/луч} \tag{14.4}$$

$$N_{\text{cone}} = \left\lfloor \frac{0.26}{\Delta\alpha} \right\rfloor \approx 15\,\text{лучей в полуконусе} \tag{14.5}$$

Итого ≈30 центральных лучей получают реальное измерение, остальные 150 — значение $r_{\max} = 3.0\,\text{м}$ (неизвестно).

14.3 Реактивная навигация (fallback)

Файл исходного кода

pi_nodes/nodes/fallback_nav_node.py (176 строк, Python)

fallback_nav_node.py, строки 122–150 — _run_fallback_drive()

```
136 if r > SAFE_M:                                  # r > 0.40 m
137     cmd['linear_x'] = FWD_SPEED                 # 0.10 m/s
138     self._state = STATE_FORWARD
139 elif r > STOP_M:                                # 0.20 < r < 0.40
140     scale = (r - STOP_M) / (SAFE_M - STOP_M)
141     cmd['linear_x'] = SLOW_SPEED + scale * (FWD_SPEED - SLOW_SPEED
        )
142     self._state = STATE_CAUTION
143 else:                                           # r < 0.20 m
144     self._state = STATE_ROTATE
145     cmd['angular_z'] = ROT_SPEED * self._rotate_dir  # +/- 0.8 rad
        /s
```

14.3.1 Пропорциональное масштабирование скорости

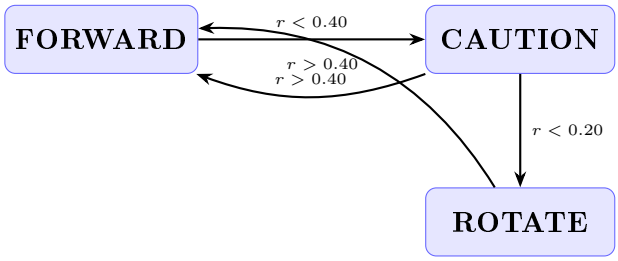
В зоне осторожности ($r_{\text{stop}} < r < r_{\text{safe}}$):

$$\text{scale} = \frac{r - r_{\text{stop}}}{r_{\text{safe}} - r_{\text{stop}}} = \frac{r - 0.20}{0.40 - 0.20} \in [0, 1] \tag{14.6}$$

$$\boxed{v = v_{\text{slow}} + \text{scale} \cdot (v_{\text{fwd}} - v_{\text{slow}})} \tag{14.7}$$

Это **линейная интерполяция** между $v_{\text{slow}} = 0.05\,\text{м/с}$ и $v_{\text{fwd}} = 0.10\,\text{м/с}$.

14.3.2 Конечный автомат реактивной навигации



14.3.3 Теоретическое обоснование: конечный автомат как дискретная система

Реактивная навигация реализует **гибридную систему** — комбинацию непрерывной динамики (скорость) и дискретных переключений (состояния автомата). В терминах ТАУ это **переключающаяся система**:

$$\dot{\boldsymbol{x}} = \boldsymbol{f}_{q(t)}(\boldsymbol{x}), \quad q \in \{\text{FWD}, \text{CAUT}, \text{ROT}\} \tag{14.8}$$

Устойчивость переключающейся системы гарантирована, если каждая подсистема устойчива и переключения не слишком частые (условие dwell time).

Линейная интерполяция (14.7) в зоне CAUTION обеспечивает **непрерывность** скорости на границах переключений, предотвращая рывки.

14.4 Сводная таблица параметров

Модуль	**Параметр**	**Значение**	**Описание**
Геозона	Разрешение	0.05 м	Размер ячейки
Геозона	Отступ	0.5 м	Запас за полигоном
LaserScan	N_{beams}	180	Кол-во лучей
LaserScan	FOV	180°	Поле зрения
Fallback	r_{safe}	0.40 м	Безопасная дистанция
Fallback	r_{stop}	0.20 м	Стоп-дистанция
Fallback	ω_{rot}	0.8 рад/с	Скорость поворота

Chapter 15

ШИМ-управление моторами и PCA9685

Файл исходного кода

ros_ws/src/robot_pkg_cpp/src/motor_node.cpp (343 строки, C++)

15.1 Контроллер PCA9685

PCA9685 — 16-канальный ШИМ-контроллер на I2C (адрес 0x5F). Каждый канал формирует ШИМ-сигнал с 12-битным разрешением (0 . . . 4095).

15.1.1 Вычисление прескейлера

motor_node.cpp, строки 91–94

```
91 // prescale = round(osc / (4096 * freq)) - 1
92 uint8_t prescale = static_cast<uint8_t>(
93   std::round(static_cast<double>(OSC_FREQ_HZ) /
94              (4096.0 * static_cast<double>(PWM_FREQ_HZ))) - 1.0);
```

Формула прескейлера для заданной частоты ШИМ:

$$\boxed{\text{prescale} = \left\lfloor \frac{f_{\text{osc}}}{4096 \cdot f_{\text{PWM}}} + 0.5 \right\rfloor - 1} \tag{15.1}$$

Для $f_{\text{osc}} = 25\,\text{МГц}$, $f_{\text{PWM}} = 50\,\text{Гц}$:

$$\text{prescale} = \left\lfloor \frac{25{,}000{,}000}{4096 \times 50} + 0.5 \right\rfloor - 1 = \lfloor 122.07 + 0.5 \rfloor - 1 = 121 \tag{15.2}$$

15.2 Slow-Decay режим управления мотором

motor_node.cpp, строки 210–232 — setMotor()

```
210 void setMotor(int idx, double throttle)
211 {
212   throttle = std::clamp(throttle, -1.0, 1.0);
213   int in1, in2;
214   if (throttle > 0.001) {
215     // Forward: IN1 = 4095,  IN2 = 4095*(1-t)
216     in1 = PWM_MAX;
217     in2 = static_cast<int>(PWM_MAX * (1.0 - throttle));
218   } else if (throttle < -0.001) {
219     // Reverse: IN1 = 4095*(1+t),  IN2 = 4095
220     in1 = static_cast<int>(PWM_MAX * (1.0 + throttle));
221     in2 = PWM_MAX;
222   } else {
223     // Active brake: IN1 = IN2 = 4095
224     in1 = PWM_MAX;
225     in2 = PWM_MAX;
226   }
227   pca_->setPwm(MOTORS[idx].in1, in1);
228   pca_->setPwm(MOTORS[idx].in2, in2);
229 }
```

15.2.1 Формулы Slow-Decay

В режиме **Slow Decay** (медленное затухание тока) оба вывода активны, что обеспечивает плавное торможение:

Вперёд ($t > 0$, throttle $\in (0, 1]$):

$$\mathrm{IN1} = 4095, \quad \mathrm{IN2} = 4095 \cdot (1 - t) \tag{15.3}$$

Назад ($t < 0$, throttle $\in [-1, 0)$):

$$\mathrm{IN1} = 4095 \cdot (1 + t), \quad \mathrm{IN2} = 4095 \tag{15.4}$$

Активное торможение ($t = 0$):

$$\mathrm{IN1} = \mathrm{IN2} = 4095 \quad \text{(обе обмотки замкнуты)} \tag{15.5}$$

15.2.2 Схема каналов моторов

Мотор	Позиция	IN1 канал	IN2 канал
M1	Правый передний	ch15	ch14
M2	Левый передний	ch12	ch13
M3	Левый задний	ch11	ch10
M4	Правый задний	ch8	ch9

15.3 Дифференциальный микшер

motor_node.cpp, строки 251–266

```
251 double lin_t = (max_lin_ > 0.0) ? (linear_  / max_lin_) : 0.0;
252 double ang_t = (max_ang_ > 0.0) ? (angular_ / max_ang_) : 0.0;
253
254 double left  = lin_t + ang_t;
255 double right = lin_t - ang_t;
256
257 double max_val = std::max({std::abs(left), std::abs(right), 1.0});
258 if (max_val > 1.0) { left /= max_val; right /= max_val; }
```

Нормализованные команды ($\ell \in [-1, 1]$):

$$\ell_{\text{lin}} = \frac{v}{v_{\max}}, \quad \ell_{\text{ang}} = \frac{\omega}{\omega_{\max}} \tag{15.6}$$

$$\text{left} = \ell_{\text{lin}} + \ell_{\text{ang}} \tag{15.7}$$

$$\text{right} = \ell_{\text{lin}} - \ell_{\text{ang}} \tag{15.8}$$

Нормализация (сохранение пропорции при насыщении):

$$m = \max(1, |\text{left}|, |\text{right}|), \quad \text{left} \leftarrow \frac{\text{left}}{m}, \quad \text{right} \leftarrow \frac{\text{right}}{m} \tag{15.9}$$

15.4 Теоретическое обоснование: электропривод

15.4.1 Модель двигателя постоянного тока

Электродвигатели робота описываются дифференциальным уравнением (гл. 1, раздел 1.6):

$$L_a \frac{dI}{dt} + R_a I = U - C_e \omega, \qquad J \frac{d\omega}{dt} = C_m I - M_{\text{нагр}} \tag{15.10}$$

где L_a, R_a — индуктивность и сопротивление якоря, I — ток, C_e — коэффициент ЭДС, C_m — коэффициент момента, J — момент инерции, $M_{\text{нагр}}$ — момент нагрузки.

Исключая ток, получаем передаточную функцию:

$$W(p) = \frac{\omega(p)}{U(p)} = \frac{K_m}{T_m T_e p^2 + T_m p + 1} \tag{15.11}$$

15.4.2 Slow-Decay и эффективный момент

В режиме Slow Decay эффективное напряжение на двигателе:

$$U_{\text{эфф}} = U_{\text{пит}} \cdot t, \quad t \in [-1, 1] \tag{15.12}$$

При активном торможении ($t = 0$, IN1 = IN2 = 4095) обмотки замкнуты, и ЭДС двигателя создаёт тормозной ток:

$$I_{\text{торм}} = \frac{C_e \omega}{R_a}, \qquad M_{\text{торм}} = C_m I_{\text{торм}} = \frac{C_e C_m}{R_a} \omega \tag{15.13}$$

Это обеспечивает плавное торможение, пропорциональное скорости (вязкое демпфирование).

15.4.3 Замкнутый контур «микшер + двигатель»

Передаточная функция от команды $(v_{\text{lin}}, \omega_{\text{ang}})$ до скоростей бортов:

$$\begin{pmatrix} V_L(p) \\ V_R(p) \end{pmatrix} = \frac{K_m}{T_m p + 1} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} \ell_{\text{lin}}(p) \\ \ell_{\text{ang}}(p) \end{pmatrix} \tag{15.14}$$

При $T_m \ll T_{\text{control}}$ система упрощается до статической модели $V_L \approx K_m(\ell_{\text{lin}} + \ell_{\text{ang}})$.

15.5 Параметры

Параметр	**Значение**	**Описание**
PCA9685_ADDR	0x5F	I2C-адрес контроллера
PWM_FREQ_HZ	50 Гц	Частота ШИМ
PWM_MAX	4095	12-бит макс. значение
WHEEL_BASE	0.17 м	Расстояние между гусеницами